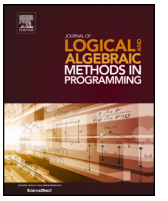
J. Log. Algebraic Methods Program. 142 (2025) 101004



Contents lists available at ScienceDirect

Journal of Logical and Algebraic Methods in Programming

journal homepage: www.elsevier.com/locate/jlamp





Comparing session type systems derived from linear logic ☆

Bas van den Heuvel [a,b,*], Jorge A. Pérez [c,*]

[a] HKA Karlsruhe, Karlsruhe, Germany
[b] University of Freiburg, Freiburg, Germany
[c] University of Groningen, Groningen, the Netherlands

ARTICLE INFO

Keywords:
Concurrency
Linear logic
π-calculus
Session types

ABSTRACT

Session types are a typed approach to message-passing concurrency, where types describe sequences of intended exchanges over channels. Session type systems have been given strong logical foundations via Curry-Howard correspondences with *linear logic*, a resource-aware logic that naturally captures structured interactions. These logical foundations provide an elegant framework to specify and (statically) verify message-passing processes.

In this paper, we rigorously compare different type systems for concurrency derived from the Curry-Howard correspondence between linear logic and session types. We address the main divide between these type systems: the classical and intuitionistic presentations of linear logic. Over the years, these presentations have given rise to separate research strands on logical foundations for concurrency; the differences between their derived type systems have only been addressed informally.

To formally assess these differences, we develop πULL, a session type system that encompasses type systems derived from classical and intuitionistic interpretations of linear logic. Based on a fragment of Girard's Logic of Unity, πULL provides a basic reference framework: we compare existing session type systems by characterizing fragments of πULL that coincide with classical and intuitionistic formulations. We analyze the significance of our characterizations by considering the *locality* principle (enforced by intuitionistic interpretations but not by classical ones) and forms of *process composition* induced by the interpretations.

1. Introduction

Establishing the correctness of *message-passing* programs is a central but challenging problem. Within formal methods for concurrency, *session types* [28,52,33] are now a consolidated approach to statically verifying safety and liveness properties of communicating programs. Session types specify communication over channels as sequences of exchanges. This way, e.g., the session type !int.?bool.end describes a channel's intended protocol: send an integer, receive a boolean, and close the channel. Due to its simplicity and expressiveness, the π-calculus [41,50]—the paradigmatic model of concurrency and interaction—is a widely used specification language for developing session types and establishing their basic theory. In this paper, we are interested in developing further the *logical foundations* of session type systems for the π-calculus.

☆ Work partially supported by the Dutch Research Council (NWO) under the VIDI Project No. 016.Vidi.189.046 (Unifying Correctness for Communicating Software).
* Corresponding authors.
E-mail addresses: vdheuvel@informatik.uni-freiburg.de (B. van den Heuvel), j.a.perez@rug.nl (J.A. Pérez).

https://doi.org/10.1016/j.jlamp.2024.101004
Received 26 January 2024; Received in revised form 18 July 2024; Accepted 10 August 2024

Available online 22 August 2024
2352-2208/© 2024 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Context: linear logic and session types In a line of work developed by Caires, Pfenning, Wadler, and several others, the theory of session types has been given firm logical foundations in the form of Curry-Howard-style correspondences. Caires and Pfenning [7] discovered a correspondence between session types for the π-calculus and Girard's *linear logic* [23]:

session types	$\Leftrightarrow$	logical propositions
π-calculus processes	$\Leftrightarrow$	proofs
process communication	$\Leftrightarrow$	cut elimination

Based on this logical bridge, the resulting type systems simultaneously ensure important properties for processes, such as *session fidelity* (processes respect session types), *communication safety* (absence of communication errors), and *progress/deadlock freedom* (processes never reach stuck states).

There are two presentations of linear logic, *classical* and *intuitionistic*, and session type systems derived from linear logic have inherited this dichotomy. Under Curry-Howard interpretations, typing judgments and typing rules correspond to the logical sequents and inference rules of the underlying linear logic. In both classical and intuitionistic cases, judgments assign independent session protocols to the channels of a process. Because these judgments differ between the presentations of linear logic, there are consequences for their respective interpretations as type systems:

- Caires and Pfenning's correspondence [7] uses an *intuitionistic* linear logic where judgments for process are two-sided, with zero or many channels on the left and exactly one channel on the right. Such judgments have a convenient rely-guarantee reading: the process *relies* on the behaviors described by the channels on the left to *guarantee* the behavior of the one channel on the right. In this interpretation, each logical connective thus requires two rules: the right rule specifies how to *offer* a behavior, while the left rule specifies how to *use* a behavior.
- Wadler's correspondence [56] uses *classical* linear logic, where judgments are single-sided (all channels appear on the right) and lack a rely-guarantee reading. In this interpretation, there is only a single rule per logical connective, which makes such type systems direct and economical.

Although the differences and relationship between intuitionistic and classical linear logic are relevant and well-known (see, e.g., [49,6,38]), the differences between their derived session type systems have only been addressed informally. In particular, Caires et al. [12] observe that, unlike classical interpretations, an intuitionistic interpretation guarantees *locality* for shared channels, i.e., the principle by which received channels cannot become servers (see Section 4.1). In the meantime, both interpretations have been extended in multiple, exciting directions (see, e.g., [9,1,13,51,39,4,8,14,53,10,34,47,19,31,32]). This emergence of *families* of type systems (one classical, one intuitionistic) has deepened the original dichotomy and somewhat obscured our understanding of logical foundations of concurrency as a whole. This state of affairs calls for a formal comparison between the session type systems derived from classical and intuitionistic linear logics that goes beyond their superficial differences. To us, this seems an indispensable step in consolidating the logical foundations of message-passing concurrency.

Goal of the paper In this paper, we aim at formally comparing the session type systems derived from interpretations of intuitionistic and classical linear logic. We are concerned with two families of type systems for the π-calculus, which naturally induce two classes of typable processes, namely those typable in intuitionistic and classical interpretations, respectively. These two classes are known to largely overlap with each other; informal observations suggest that some processes are typable in a classical setting but not in an intuitionistic setting. We are interested in determining precisely *how* these classes of processes relate to each other.

The key step in our approach is to define a basic framework of reference in which both type systems, intuitionistic and classical, can be objectively compared. Girard's Logic of Unity (LU) [24] has a similar goal: to objectively compare classical logic, intuitionistic logic, and (classical and intuitionistic) linear logic in a single system, abstracting away from syntactical differences. It is then only natural to use LU as a reference for our comparison. We develop πULL, a session type system for the π-calculus that subsumes classical and intuitionistic interpretations of linear logic as session types. Based on a fragment of LU that suits our purposes (dubbed ULL), the type system πULL allows us to define a class of typable processes that contains processes typable in both the intuitionistic and classical type systems.

In our approach, the type system πULL provides an effective framework and a yardstick for a formal comparison. Because its typing rules encompass both classical and intuitionistic formulations, we can readily characterize their two (sub-)classes of typable processes by considering the different typing rules that apply in each case. Analyzing these different portions of rules allows us to then characterize the differences (in expressiveness/typability) between the intuitionistic and classical type systems. Moreover, these characterizations allow us to determine how extensions to the type systems might be supported by the intuitionistic and classical interpretations.

Contributions and outline Summing up, this paper makes the following contributions:

1. The session type system πULL, which is derived from a concurrent interpretation of ULL (a linear fragment of LU), together with its correctness results (Theorems 2.1 to 2.4, § 2);
2. A formal comparison between (i) the classes of processes typable under interpretations of intuitionistic and classical linear logic and (ii) those typable in πULL (§ 3). We prove that (1) πULL *precisely captures* the class of processes typable under the classical

interpretation (Theorem 3.1), and that (2) the class of processes typable under the intuitionistic interpretation is *strictly included* in πULL (Theorem 3.3). Together, these two results confirm the informal observation that the two interpretations induce different classes of processes.

3. An analysis of the significance of our characterizations in terms of two different aspects: the *locality* principle enforced by intuitionistic interpretations, and *process composition* usually limited in type systems derived from linear logic in comparison to session type systems not based on logical foundations (§ 4). This analysis corroborates and gives a formal footing to the informal observation made in [12] concerning locality, and suitably extends it to show that intuitionistic interpretations cannot type empty sends on previously received channels.

Finally, we draw some conclusions in Section 5.

Related work Our work can be seen as an expressiveness study on the classes of processes induced by different type systems. Concretely, our results can be seen as providing formal evidence that session type systems derived from classical linear logic are *more expressive* than those based on intuitionistic linear logic, based on the fact that the former are *more permissive* than the latter.

The study of the relative expressiveness of process calculi has a long, fruitful history (see, e.g. [26,45,43]). The aim is to compare two different process languages by defining an *encoding* (a translation up to some correctness criteria) between them or by proving its non-existence. Even though our aims are similar in spirit to expressiveness studies, at a technical level there are substantial differences, because our focus is on assessing the influence that different type systems have on the same process language—as such, our comparisons do not rely on encodings.

Although most studies on relative expressiveness have addressed *untyped* process languages, some works have studied the influence of (behavioral) types on the expressiveness of process languages—see, e.g., [16,22,15,37,44]. Salient examples are the works [18,46], which compare type systems that enforce the deadlock-freedom and termination properties, respectively. In particular, a main result in [18] is that (classical) linear logic induces a strict subclass of deadlock-free processes with respect to the class of typable processes in non-logically motivated type systems. Unlike our work, the focus in [18] is on different process languages, each with a different type system. This requires the definition of encodings, on processes but also on types, that abstract away from the syntactic differences between the classes of processes induced by each typed framework.

This paper is an extended, revised version of the workshop paper [29]. We present several improvements and developments with respect to that work. First, here we have significantly improved the correctness results for πULL (Theorems 2.1 to 2.4) and considered branching/selection types (not studied in [29]). Also, we have refined the presentation of πULL in [29] with a more explicit treatment of duality, presented in Section 2.3. Moreover, we have broadened the analysis of our comparison results in Section 4 by considering extensions to the type system with more expressive forms of parallel composition and restriction.

2. A session type system based on LU

Girard [24] developed Logic of Unity (LU) to study and compare classical, intuitionistic, and linear logic, without having to "change the rules of the game by, e.g., passing from one style of sequent to the other." The idea is simple: there is one form of sequent with an abundance of inference rules. The several logics subsumed by LU are then characterized by (possibly overlapping) subsets of those rules.

Clearly, we find ourselves in a similar situation: we want to compare intuitionistic and classical linear logic as session type systems, by abstracting away from typing judgments and rules of different forms. To this end, in this section we introduce United Linear Logic (ULL), a logic based on the linear fragment of LU, and present the Curry-Howard interpretation of ULL as a session type system for the π-calculus, dubbed πULL, following [7,56] (§ 2.1). As we will see in Section 3, we can then characterize the session type interpretations of intuitionistic and classical linear logic as subsets of the typing rules of πULL. We also present correctness properties common to (logically motivated) session type systems (§ 2.2). Finally, we discuss an alternative presentation of πULL that has a more explicit account of duality (§ 2.3).

2.1. The process calculus and type system

Propositions/types A session type represents a sequence of exchanges that should be performed along a channel. Propositions in ULL—interpreted as session types in πULL—are defined as follows:

Definition 2.1. ULL *propositions/*πULL *types* are generated by the following grammar:

$$A, B ::= \mathbf{1} \mid \bot \mid A \otimes B \mid A \multimap B \mid \oplus\{i : A\}_{i\in I} \mid \&\{i : A\}_{i\in I} \mid !A \mid ?A$$

Table 1 gives the intuitive reading of the interpretation of propositions as session types. Some details are worth noting.

- First, receiving in πULL is defined using the intuitionistic connective $\multimap$; we will see later that $\multimap$ can be used to define the classical connective $\parr$—also interpreted as receiving—which is not supported by intuitionistic linear logic.

Table 1
Interpretation of ULL propositions as session types.

$\mathbf{1}$ and $\bot$	Close the channel
$A \otimes B$	Send a channel of type A and continue as B
$A \multimap B$	Receive a channel of type A and continue as B
$\oplus\{i : A_i\}_{i\in I}$	Send a label $i \in I$ and continue as A_i
$\&\{i : A_i\}_{i\in I}$	Receive a label $i \in I$ and continue as A_i
$!A$	Repeatedly provide a service of type A
$?A$	Connect to a service of type A

- Second, πULL supports labeled n-ary branching constructs (following, e.g., Caires and Pérez in [8]). In linear logic, $\oplus$ and & are binary connectives [23,25,24]. In [7,56], Caires, Pfenning, and Wadler originally interpret them as a choice between a left and a right option. However, in session-typed π-calculi (see, e.g., [33]) n-ary choice is standard and more convenient.
- Third, the syntax of formulas considered by Girard [24] subsumes our syntax for ULL by including universal and existential quantifiers and the units $\top$ and $\mathbf{0}$. Our syntax does not include quantifiers and considers both $\mathbf{1}$ and $\bot$: while the interpretation of intuitionistic linear logic by Caires and Pfenning includes $\mathbf{1}$ (but not $\top$, $\mathbf{0}$, or $\bot$), the interpretation of classical linear logic by Wadler considers quantifiers and the units $\mathbf{1}$ and $\bot$ (but not $\top$ or $\mathbf{0}$).

Duality The duality of ULL propositions is defined as follows:

Definition 2.2. *Duality* is a binary relation on propositions/types, denoted $A^\perp$, defined as follows:

$$\mathbf{1}^\perp := \bot \qquad (A \otimes B)^\perp := A \multimap B^\perp \qquad (\oplus\{i : A_i\}_{i\in I})^\perp := \&\{i : A_i^\perp\}_{i\in I} \qquad (!A)^\perp := ?A^\perp$$
$$\bot^\perp := \mathbf{1} \qquad (A \multimap B)^\perp := A \otimes B^\perp \qquad (\&\{i : A_i\}_{i\in I})^\perp := \oplus\{i : A_i^\perp\}_{i\in I} \qquad (?A)^\perp := !A^\perp$$

Duality in πULL reflects the intended reciprocity of protocols between two parties: when a process on one side of a channel sends, the process on the opposite side must receive, and vice versa. It is easy to see that duality is an involution: $(A^\perp)^\perp = A$.

As mentioned before, we can use duality to define $\parr$ in terms of $\multimap$: $A \parr B := A^\perp \multimap B$; as we will see, the rules for $\parr$ of classical linear logic can be recovered from the rules for $\multimap$ in ULL using duality. Duality also shows that $\otimes$ and $\parr$ are De Morgan-style duals in classical linear logic:

$$(A \otimes B)^\perp = A \multimap B^\perp = (A^\perp)^\perp \multimap B^\perp = A^\perp \parr B^\perp$$
$$(A \parr B)^\perp = (A^\perp \multimap B)^\perp = A^\perp \otimes B^\perp$$

Processes The discipline of binary session types deals with concurrent *processes* that communicate through point-to-point *channels*. The π-calculus [41,50] offers a rigorous yet expressive framework for defining this discipline and establishing its fundamental properties. πULL is a type system for π-calculus processes defined as follows:

Definition 2.3. *Process terms* are generated by the following grammar:

$$P, Q ::= \mathbf{0} \mid (\nu x)P \mid P \mid Q \mid x\langle y\rangle.P \mid x(y).P \mid x \lhd \ell.P \mid x \rhd \{i : P\}_{i\in I}$$
$$\mid\ !x(y).P \mid [x \leftrightarrow y] \mid x\langle\rangle.P \mid x().P$$

Process constructs for inaction $\mathbf{0}$, channel restriction $(\nu x)P$, and parallel composition $P \mid Q$ have standard readings. The same applies to constructs for sending, receiving, selection, branching, and replicated receive prefixes, which are respectively denoted $x\langle y\rangle.P$, $x(y).P$, $x \lhd \ell.P$, $x \rhd \{i : P_i\}_{i\in I}$, and $!x(y).P$. Note that all these prefixes are *blocking*, which means that our process calculus implements *synchronous* communication (see, e.g., [13,30,55] for interpretations of linear logic as session type systems in the asynchronous setting). Process $[x \leftrightarrow y]$ denotes a forwarder that "fuses" channels x and y; it is akin to the *link processes* used in encodings of name passing using internal mobility [2].

We consider also constructs $x\langle\rangle.P$ and $x().P$, which specify the explicit closing of channels: their synchronization represents the explicit deallocation of linear resources. We use these constructs to give a *non-silent* interpretation of $\mathbf{1}$ and $\bot$ (see § 3.1).

In $(\nu y)P$, $x(y).P$, and $!x(y).P$ the occurrence of y is binding, with scope P; we say that y is *bound* in P. Non-bound occurrences of names are considered *free*. We consistently adopt Barendregt's naming convention: bound names are pairwise distinct and distinct from free names. The sets of bound and free names of a process P are denoted $\mathrm{bn}(P)$ and $\mathrm{fn}(P)$, respectively. We identify processes up to consistent renaming of bound names, writing $\equiv_\alpha$ for this congruence. We write $P\{y/x\}$ for the capture-avoiding substitution of free occurrences of y for x in P.

Structural congruence The following is an important notation of syntactical equivalence for πULL processes.

$$[\text{cutSymm}]\ \frac{}{(\nu x)(P \mid Q) \equiv (\nu x)(Q \mid P)} \qquad [\text{cutAssocL}]\ \frac{x \notin \text{fn}(Q) \qquad y \notin \text{fn}(P)}{(\nu x)(P \mid (\nu y)(Q \mid R)) \equiv (\nu y)(Q \mid (\nu x)(P \mid R))}$$

$$[\text{cutAssocR}]\ \frac{x \notin \text{fn}(R) \qquad y \notin \text{fn}(P)}{(\nu x)(P \mid (\nu y)(Q \mid R)) \equiv (\nu y)((\nu x)(P \mid Q) \mid R)}$$

$$[\beta\text{ID}]\ \frac{x \neq y}{(\nu x)(P \mid [x \leftrightarrow y]) \to P\{y/x\}} \qquad [\beta\text{CLOSE}]\ \frac{}{(\nu x)(x\langle\rangle.\mathbf{0} \mid x().Q) \to Q}$$

$$[\beta\text{SEND}]\ \frac{}{(\nu x)((\nu y)x\langle y\rangle.(P_1 \mid P_2) \mid x(z).Q) \to (\nu x)(P_2 \mid (\nu y)(P_1 \mid Q\{y/z\}))}$$

$$[\beta\text{SEL}]\ \frac{j \in I}{(\nu x)(x \lhd j.P \mid x \rhd \{i : Q_i\}_{i \in I}) \to (\nu x)(P \mid Q_j)}$$

$$[\beta\text{SERV}]\ \frac{}{(\nu x)((\nu y)x\langle y\rangle.P \mid !x(z).Q) \to (\nu x)((\nu y)(P \mid Q\{y/z\}) \mid !x(z).Q)} \qquad [\beta\text{WEAKEN}]\ \frac{x \notin \text{fn}(P)}{(\nu x)(P \mid !x(z).Q) \to P}$$

$$[\kappa\text{CLOSE}]\ \frac{}{(\nu y)(P \mid x().Q) \to x().(\nu y)(P \mid Q)}$$

$$[\kappa\text{sendR}]\ \frac{y \in \text{fn}(Q_2)}{(\nu y)(P \mid (\nu z)x\langle z\rangle.(Q_1 \mid Q_2)) \to (\nu z)x\langle z\rangle.(Q_1 \mid (\nu y)(P \mid Q_2))}$$

$$[\kappa\text{sendL}]\ \frac{y \in \text{fn}(Q_1)}{(\nu y)(P \mid (\nu z)x\langle z\rangle.(Q_1 \mid Q_2)) \to (\nu z)x\langle z\rangle.((\nu y)(P \mid Q_1) \mid Q_2)}$$

$$[\kappa\text{RECV}]\ \frac{}{(\nu y)(P \mid x(z).Q) \to x(z).(\nu y)(P \mid Q)} \qquad [\kappa\text{SEL}]\ \frac{}{(\nu y)(P \mid x \lhd \ell.Q) \to x \lhd \ell.(\nu y)(P \mid Q)}$$

$$[\kappa\text{BRA}]\ \frac{}{(\nu y)(x \rhd \{i : P_i\}_{i \in I} \mid Q) \to x \rhd \{i : (\nu y)(P_i \mid Q)\}_{i \in I}}$$

Fig. 1. Structural congruence and reduction for πULL.

Definition 2.4. *Structural congruence* is a binary relation on process terms, denoted $P \equiv Q$. It is defined as the least congruence on processes (i.e., closed under arbitrary process contexts) that satisfies the axioms in Fig. 1 (top).

The above formulation of structural congruence internalizes the conditions induced by typing (i.e., proof transformations in ULL), necessary for the correctness properties proven in Section 2.2.

Computation As is usual for Curry-Howard correspondences, computation is related to cut reduction. Cuts are used in logic to combine two proofs that contain dual propositions. As we will see, a cut in session type interpretations of linear logic entails the parallel composition and connection of two processes. Generally, cut reduction transforms cuts into cuts on smaller types. In correspondences between linear logic and session types, cut reduction corresponds to *communication*—the notion of computation in the π-calculus, which expresses internal behavior of processes.

The definition of the reduction relation follows. Note that, for a simpler presentation, it includes a set of rules for commuting conversions (κ-rules); an alternative presentation could treat commuting conversions as a behavioral equivalence.

Definition 2.5. *Reduction* is a binary relation on process terms, denoted $P \to Q$, defined in Fig. 1. It is closed under the following rules:

$$\frac{Q \to Q'}{P \mid Q \to P \mid Q'}\ [\text{par}] \qquad \frac{P \to Q}{(\nu y)P \to (\nu y)Q}\ [\text{res}] \qquad \frac{P \equiv P' \qquad P' \to Q' \qquad Q' \equiv Q}{P \to Q}\ [\text{sc}]$$

We write $\to_\beta$ to denote reductions that follow from β-rules.

In this definition, Rule [βID] replaces a channel x with a channel y which x is forwarded to. Rule [βCLOSE] formalizes the explicit channel deallocation mentioned above: the synchronization between an empty send and an empty receive effectively closes channel x.

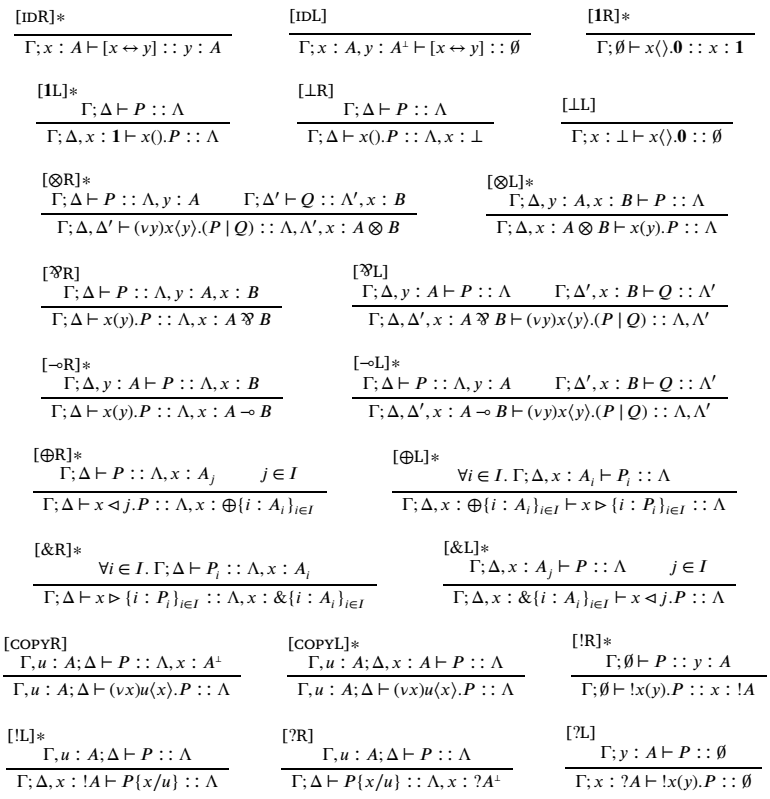


Fig. 2. The πULL type system.

Rule [βSEND] formalizes the synchronization between a send and a corresponding receive, substituting the sent name for the received name in the continuation of the receive. Rule [βSEL] formalizes the synchronization between a selection and a branch. Rule [βSERV] allows to synchronize a client and a service; a copy of the service Q is spawned with the sent name substituted for the received name while the replicated receive remains. Rule [βWEAKEN] allows cleaning up a service if it has no clients.

The κ-rules reflect *commuting conversions* in linear logic, which in the process calculus allow pulling prefixes on free names out of series of consecutive restrictions; as we will see, we use them such that we can state a general progress result.

Rules [PAR], [RES], and [SC] close reduction under parallel composition, restriction and structural congruence, respectively (Definition 2.4).

Type checking The inference system of ULL is a sequent calculus with sequents of the form $\Gamma; \Delta \vdash \Lambda$. Here, Γ, Δ and Λ denote *regions* which collect propositions and obey different structural criteria. Γ is the *unrestricted* region, which contains propositions that can be indefinitely used. Δ and Λ are the *linear* regions, which contain propositions that must be used exactly once. We write '$\emptyset$' to denote an empty region. Also, we extend duality to regions: $(\Delta)^{\perp}$ contains exactly the duals of the propositions in Δ.

The type system for πULL is an extension of ULL's inference system with process and channel name annotations on sequents, such that judgments are of the form $\Gamma; \Delta \vdash P :: \Lambda$. Then, the regions Γ, Δ, and Λ denote the unrestricted resp. linear *contexts* of P, consisting of assignments $x : A$ where x is a channel name and A is a proposition/type.

Figs. 2 and 3 give the typing rules of πULL, which are based directly on the linear fragment of LU in [24] (some rules are marked with $*$, which we will refer to later).

We comment on the rules in Fig. 2. Axioms [IDR] and [IDL] type forwarding constructs which connect two channels of dual type. Axioms [**1**R] and [$\perp$L] type processes that close a session with an empty send after which they become inactive. Rules [$\perp$R] and [**1**L] type processes that close a session with an empty receive. These four rules define a *non-silent* interpretation for **1** and $\perp$ that entails process communication (cf. Definition 2.5), which corresponds to cut reductions in proofs. (An alternative *silent* interpretation of **1** and $\perp$ is discussed in Section 4.2.1.)

The typing system elegantly induces processes under the *internal* mobility discipline, whereby only fresh channels are exchanged in communications [48,2]. Rules [$\otimes$R], [$\parr$L], and [$\multimap$L] type bound sends, where one process provides the sent channel and another independent process provides the continuation channel. Rules [$\otimes$L], [$\parr$R], and [$\multimap$R] type receive-prefixed processes. Rules [$\oplus$R] and [&L] type selection and rules [$\oplus$L] and [&R] type branching.

$$[\text{CUTRL}]* \quad \frac{\Gamma;\Delta \vdash P :: \Lambda, x : A \qquad \Gamma;\Delta', x : A \vdash Q :: \Lambda'}{\Gamma;\Delta,\Delta' \vdash (\nu x)(P \mid Q) :: \Lambda,\Lambda'}$$

$$[\text{CUTLR}]* \quad \frac{\Gamma;\Delta, x : A \vdash P :: \Lambda \qquad \Gamma;\Delta' \vdash Q :: \Lambda', x : A}{\Gamma;\Delta,\Delta' \vdash (\nu x)(P \mid Q) :: \Lambda,\Lambda'}$$

$$[\text{CUTRR}] \quad \frac{\Gamma;\Delta \vdash P :: \Lambda, x : A \qquad \Gamma;\Delta' \vdash Q :: \Lambda', x : A^{\perp}}{\Gamma;\Delta,\Delta' \vdash (\nu x)(P \mid Q) :: \Lambda,\Lambda'}$$

$$[\text{CUTLL}] \quad \frac{\Gamma;\Delta, x : A \vdash P :: \Lambda \qquad \Gamma;\Delta', x : A^{\perp} \vdash Q :: \Lambda'}{\Gamma;\Delta,\Delta' \vdash (\nu x)(P \mid Q) :: \Lambda,\Lambda'}$$

$$[\text{CUT!R}]* \quad \frac{\Gamma, u : A;\Delta \vdash P :: \Lambda \qquad \Gamma;\emptyset \vdash Q :: x : A}{\Gamma;\Delta \vdash (\nu u)(P \mid !u(x).Q) :: \Lambda}$$

$$[\text{CUT!L}]* \quad \frac{\Gamma;\emptyset \vdash P :: x : A \qquad \Gamma, u : A;\Delta \vdash Q :: \Lambda}{\Gamma;\Delta \vdash (\nu u)(!u(x).P \mid Q) :: \Lambda}$$

$$[\text{CUT?R}] \quad \frac{\Gamma, u : A;\Delta \vdash P :: \Lambda \qquad \Gamma;x : A^{\perp} \vdash Q :: \emptyset}{\Gamma;\Delta \vdash (\nu u)(P \mid !u(x).Q) :: \Lambda}$$

$$[\text{CUT?L}] \quad \frac{\Gamma;x : A^{\perp} \vdash P :: \emptyset \qquad \Gamma, u : A;\Delta \vdash Q :: \Lambda}{\Gamma;\Delta \vdash (\nu u)(!u(x).P \mid Q) :: \Lambda}$$

Fig. 3. Cut-rules of the πULL type system.

Our interpretation of $!A$ and $?A$ as server and client behaviors follows the interpretation of classical linear logic in [12]. Rules [COPYR] and [COPYL] type clients that connect to a service by sending a fresh channel. Rules [!R] and [?L] allow the typing of unused services and Rules [!L] and [?R] allow adding unused services to the unrestricted context.

Fig. 3 gives a series of so-called *cut-rules*, that type channel connections. The number and shape of these rules is a difference with respect to previous presentations. Rules [CUTRL], [CUTLR], [CUTLL], and [CUTRR] type pairs of processes that have a channel of dual type in common by composing them in parallel and immediately binding their common channel. The four similar rules provide for all possible sides the cut channel can appear on. This way, constructs for restriction and parallel composition are jointly treated. Rules [CUT!R], [CUT!L], [CUT?R], and [CUT?R] type the connection of a service provider with potential clients; a process Q with potential clients has a channel u in its unrestricted context, so the rules create a service from a process P that has a single channel x of type dual to u's type by prefixing it with replicated reception on u (forming $!u(x).P$) and then composing this process in parallel with Q and binding u.

This abundance of cut-rules is derived from the generality of ULL's judgments and necessary for proving the correctness results presented in Section 2.2. In Section 2.3 we shall consider an alternative presentation of πULL, which allows moving channels between the left- and right-hand regions of typing judgments using duality; as we will see, in such a presentation we will be able to drastically cut down the number of cut-rules.

Differences with LU As already mentioned, for the purposes of our formal comparison we consider a linear logic derived from LU [24] restricted to linear connectives. The following are notable differences between our linear logic and the linear fragment of LU:

- we include a Rule [IDL] which is complementary to [IDR];
- we include Rules [**1**L] and [$\perp$R] which are lacking in LU;
- we omit rules for $\top$ and **0** (the units of & and $\oplus$, resp.), which are usually disregarded in session type interpretations of linear logic (an exception is [31], which uses $\top$ and **0** to give a logical account of subtyping);
- we omit rules that move propositions between the left and right linear regions using duality (in Section 2.3 we will return to these rules);
- because the order of assumptions in typing rules makes a practical difference, we include additional symmetric cut-rules.

2.2. Correctness properties

Session type systems for the π-calculus derived from the Curry-Howard correspondence enforce strong correctness properties for processes, which follow directly from properties of the logic, in particular from cut elimination. This is no different for πULL. Our first result is the *safety* property of *subject congruence and reduction* (Theorems 2.1 and 2.2), which says that typability is consistent across structural congruence and reductions.

Theorem 2.1 *(Subject congruence). If* $\Gamma;\Delta \vdash P :: \Lambda$ *and* $P \equiv Q$*, then* $\Gamma;\Delta \vdash Q :: \Lambda$.

Proof (Sketch). By induction on the derivation of the structural congruence. The only inductive case is the closure under arbitrary process contexts, which follows from the IH directly. The base cases correspond to a rule in Fig. 1 (top). In each case, we infer the typing of P and Q from the shapes of the processes in the rule, and show that these typing inferences have identical assumptions and conclusion. The cases of Rules [CUTASSOCL] and [CUTASSOCR] are straightforward as usual. The analysis of the more interesting case of Rule [CUTSYMM] depends on the last-applied cut-rule. If the left-hand side uses, e.g., Rule [CUTLR], then the right-hand side should use Rule [CUTRL]. □

Theorem 2.2 *(Subject reduction). If* $\Gamma;\Delta \vdash P :: \Lambda$ *and* $P \to Q$*, then* $\Gamma;\Delta \vdash Q :: \Lambda$.

Proof (Sketch). By induction on the derivation of the reduction. The cases correspond to the rules in Fig. 1 (bottom), as well as the closure rules in Definition 2.5. In each case, we infer the typing of P and construct one for Q from the shapes of the processes in the rule, and show that these typing inferences have identical assumptions and conclusions. We detail two cases.

The case of Rule [βSERV] serves to illustrate the need for multiple symmetric cut-rules. Suppose, for example, that the last-applied rule is Rule [CUT!R], and that the client request is derived using Rule [COPYL]:

$$\dfrac{\dfrac{\Gamma, x : A; \Delta, y : A \vdash P :: \Lambda}{\Gamma, x : A; \Delta \vdash (\nu y)x\langle y\rangle.P :: \Lambda}\,[\text{COPYL}] \qquad \Gamma; \emptyset \vdash Q :: z : A}{\Gamma; \Delta \vdash (\nu x)((\nu y)x\langle y\rangle.P \mid !x(z).Q) :: \Lambda}\,[\text{CUT!R}]$$

As per Rule [βSERV], we need to identically type $(\nu x)((\nu y)(P \mid Q\{y/z\}) \mid !x(z).Q)$ using the same assumptions as above. Had we only had, e.g., Rules [CUTRL] and [CUTLL], this would not be possible. However, with the rules in Fig. 3 it is no problem. We first have to add $x : A$ into the persistent regions of the proof of the typing of Q, and substitute y for z, after which we derive the following:

$$\dfrac{\dfrac{\Gamma, x : A; \Delta, y : A \vdash P :: \Lambda \qquad \Gamma, x : A; \emptyset \vdash Q\{y/z\} :: y : A}{\Gamma, x : A; \Delta \vdash (\nu y)(P \mid Q\{y/z\}) :: \Lambda}\,[\text{CUTLR}] \qquad \Gamma; \emptyset \vdash Q :: z : A}{\Gamma; \Delta \vdash (\nu x)((\nu y)(P \mid Q\{y/z\}) \mid !x(z).Q) :: \Lambda}\,[\text{CUT!R}]$$

As another representative case, we consider Rule [βSEND]. We have $P = (\nu x)((\nu y)x\langle y\rangle.(R \mid S) \mid x(z).T) \to (\nu x)(S \mid (\nu y)(R \mid T\{y/z\})) = Q$. There are multiple ways to type P, depending on the cut-rule applied. Here, we give the example of Rule [CUTR]. The proof of $\Gamma; \Delta \vdash P :: \Lambda$ looks as follows:

$$\dfrac{\dfrac{\Gamma; \Delta_1 \vdash R :: \Lambda_1, y : A \qquad \Gamma; \Delta_2 \vdash S :: \Lambda_2, x : B}{\Gamma; \Delta_1, \Delta_2 \vdash (\nu y)x\langle y\rangle.(R \mid S) :: \Lambda_1, \Lambda_2, x : A \otimes B}\,[\otimes\text{R}] \qquad \dfrac{\Gamma; \Delta_3, z : A, x : B \vdash T :: \Lambda_3}{\Gamma; \Delta_3, x : A \otimes B \vdash x(z).T :: \Lambda_3}\,[\otimes\text{L}]}{\Gamma; \underbrace{\Delta_1, \Delta_2, \Delta_3}_{\Delta} \vdash \underbrace{(\nu x)((\nu y)x\langle y\rangle(R \mid S) \mid T)}_{P} :: \underbrace{\Lambda_1, \Lambda_2, \Lambda_3}_{\Lambda}}\,[\text{CUTR}]$$

We can then construct a proof of $\Gamma; \Delta \vdash Q :: \Lambda$ using the assumptions in the above proof.

$$\dfrac{\Gamma; \Delta_2 \vdash S :: \Lambda_2, x : B \qquad \dfrac{\Gamma; \Delta_1 \vdash R :: \Lambda_1, y : A \qquad \Gamma; \Delta_3, y : A, x : B \vdash T\{y/z\} :: \Lambda_3}{\Gamma; \Delta_1, \Delta_3, x : B \vdash (\nu y)(R \mid T\{y/z\}) :: \Lambda_1, \Lambda_3}\,[\text{CUTR}]}{\Gamma; \Delta \vdash \underbrace{(\nu x)(S \mid (\nu y)(R \mid T\{y/z\}))}_{Q} :: \Lambda}\,[\text{CUTR}] \qquad \square$$

Our next result is *progress*, the *safety* property that says that the specific form of composition and restriction in πULL following from the cut-rule enables communication, and that processes never get stuck waiting for each other:

Theorem 2.3 *(Progress). If $\Gamma; \Delta \vdash P :: \Lambda$ and $P \equiv (\nu x)(Q \mid R)$, then there exists P' such that $P \to P'$.*

Proof. By induction on the size of the proof of $\Gamma; \Delta \vdash P :: \Lambda$. By Theorem 2.1, $\Gamma; \Delta \vdash (\nu x)(Q \mid R) :: \Lambda$. By assumption, the last inference of the derivation thereof is either a linear cut or an unrestricted cut.

(Case linear cut) The last-applied rule can be [CUTR] or [CUTL]. W.l.o.g. assume the former. By inversion of [CUTR], we have a proof π_Q of $\Gamma; \Delta_Q \vdash Q :: \Lambda_Q, x : A$ and a proof π_R of $\Gamma; \Delta_R, x : A \vdash R :: \Lambda_R$ where $\Delta_Q, \Delta_R = \Delta$ and $\Lambda_Q, \Lambda_R = \Lambda$.

If the last-applied rules in π_Q and π_R are both on x, then we apply a β-reduction depending on A. For example, assume $A = B \otimes C$. Then the last-applied rules in π_Q and π_R are [$\otimes$R] and [$\otimes$L], respectively. Hence, by Rule [βSEND], $P \equiv (\nu x)((\nu y)x\langle y\rangle.(Q_y \mid Q_x) \mid x(y).R') \to (\nu x)(Q_x \mid (\nu y)(Q_y \mid R'))$.

Otherwise, w.l.o.g. assume the last-applied rule not on x is in π_Q. Then, if Q is a cut, by the induction hypothesis, $Q \to Q'$, and hence $P \equiv (\nu x)(Q \mid R) \to (\nu x)(Q' \mid R)$. Otherwise, Q is prefixed by an action on some free channel y which is not a free channel of R. Hence, we apply a κ-conversion depending on the type of the channel the last-applied rule in π_Q works on. For example, if this rule introduces $y : B \otimes C$ on the right, then, by Rule [$\kappa\otimes$], $P \equiv (\nu x)((\nu z)y\langle z\rangle.(Q_y \mid Q_{z,x}) \mid R_x) \to (\nu z)y\langle z\rangle.((\nu x)(Q_{z,x} \mid R_x) \mid Q_y)$.

(Case unrestricted cut) The last-applied rule can be [CUT!] or [CUT?]. W.l.o.g. assume the former. Then $Q \equiv !x(y).Q'$, and, by inversion of this rule, we have a proof $\pi_{Q'}$ of $\Gamma; \emptyset \vdash Q' :: y : A$ and a proof π_R of $\Gamma, x : A; \Delta \vdash R :: \Lambda$. If $x \notin \text{fn}(R)$, then, by Rule [βWEAKEN], $P \equiv (\nu x)(R \mid !x(y).Q') \to R$. Otherwise, the next step depends on the last-applied rule in π_R.

If the last-applied rule in π_R is on x, then it must be [COPYR] or [COPYL]. W.l.o.g. assume the former. Then $R \equiv (\nu y)x\langle y\rangle.R'$, so, by Rule [$\beta$SERV], $P \equiv (\nu x)((\nu y)x\langle y\rangle.R' \mid !x(y).Q') \to (\nu x)((\nu y)(Q' \mid R') \mid !x(y).Q')$.

Otherwise, the proof proceeds as in the last part of the case of linear cut. $\square$

It follows from progress that *closed* processes are *deadlock free*. In a closed process, all channels are connected, except for one channel. This remaining channel, which the process simply closes, can be seen as a *barb* that signals successful termination. As such, deadlock freedom ensures that closed processes that cannot reduce are successfully terminated and thus not stuck in a deadlock. Let us write $P \not\to$ to denote that there is no P' such that $P \to P'$.

Theorem 2.4 *(Deadlock freedom). Suppose* $\emptyset;\emptyset \vdash P :: z : \mathbf{1}$ *or* $\emptyset; z : \bot \vdash P :: \emptyset$. *If* $P \not\rightarrow$, *then* $P \equiv z\langle\rangle.\mathbf{0}$.

Proof. Towards contradiction, assume $P \not\equiv z\langle\rangle.\mathbf{0}$. The only well-typed possibility is that $P \equiv (\nu x)(Q \mid R)$. By Theorem 2.3, then $P \rightarrow P'$. This contradicts the assumption that $P \not\rightarrow$, proving the thesis. □

The combination of Theorems 2.2 and 2.3 allows us to adapt existing techniques (e.g., the logical relations of [42]) to prove *termination*: the *liveness* property that says that well-typed processes eventually stop reducing.

2.3. On duality

It may seem that there is an extensive redundancy in the system of rules in Fig. 2, caused by the two sidedness of πULL's judgments: every connective can be inferred on either side of judgments. For example, Rules [⊗R], [⅋L], and [⊸L] all type the send of a channel; which rule to use depends on the side of the judgment the involved channels are on. However, there is no actual redundancy, for if we were to omit rules for, e.g., ⅋ and ⊸, it would be impossible to type a send on a previously received channel.

This abundance of typing rules in πULL can be explained by its full support for duality: for every rule inferring a connective on one side of a judgment, there is a rule for inferring the connective's dual on the other side of a judgment. To make this duality explicit, we define an alternative type system by restricting πULL's rules to a specific fragment and adding LU's rules for *moving propositions* between sides of judgments:

Definition 2.6. The type system $\pi\mathsf{ULL}_{\curvearrowright}$, with judgments $\Gamma;\Delta \vdash_{\curvearrowright} P :: \Lambda$, is defined on the process calculus as defined in Section 2.1. Its rules are the $*$-marked rules in Fig. 2 plus the following rules:

$$\frac{\Gamma;\Delta \vdash_{\curvearrowright} P :: \Lambda, x : A}{\Gamma;\Delta, x : A^{\perp} \vdash_{\curvearrowright} P :: \Lambda}\ [\curvearrowleft] \qquad \frac{\Gamma;\Delta, x : A \vdash_{\curvearrowright} P :: \Lambda}{\Gamma;\Delta \vdash_{\curvearrowright} P :: \Lambda, x : A^{\perp}}\ [\curvearrowright]$$

Fortunately, in the presence of these two rules, a number of other rules become truly redundant: all rules in Fig. 2 not marked with $*$ are admissible or derivable in $\pi\mathsf{ULL}_{\curvearrowright}$. Dually, Rules [↶] and [↷] are admissible in vanilla πULL. The following theorem formalizes these facts:

Theorem 2.5.

1. *The rules in Figs. 2 and 3 not marked with* $*$ *are admissible or derivable in* $\pi\mathsf{ULL}_{\curvearrowright}$, *and*
2. *Rules* [↶] *and* [↷], *as given in Definition 2.6, are admissible in* πULL.

Proof. *(Item 1)* Suppose given a proof of $\Gamma;\Delta \vdash_{\curvearrowright} P :: \Lambda$. By applying induction on the structure of this proof we show that any applications of non-$*$-marked rules can be replaced with applications of $*$-marked rules in combination with uses of Rules [↶] or [↷]. We discuss every possible last-applied rule, omitting cases of $*$-marked rules as they follow directly from the induction hypothesis.

• [IDL]	1	$\Gamma;x : A, y : A^{\perp} \vdash_{\curvearrowright} [x \leftrightarrow y] :: \emptyset$	(assumption)
	2	$\Gamma;x : A \vdash_{\curvearrowright} [x \leftrightarrow y] :: y : A$	([IDR])
	3	$\Gamma;x : A, y : A^{\perp} \vdash_{\curvearrowright} [x \leftrightarrow y] :: \emptyset$	([↶] on 2)
• [⊥R]	1	$\Gamma;\Delta \vdash_{\curvearrowright} x().P :: \Lambda, x : \bot$	(assumption)
	2	$\Gamma;\Delta \vdash_{\curvearrowright} P :: \Lambda$	(inversion on 1)
	3	$\Gamma;\Delta \vdash_{\curvearrowright} P :: \Lambda$ with only $*$ rules	(IH on 2)
	4	$\Gamma;\Delta, x : \mathbf{1} \vdash_{\curvearrowright} P :: \Lambda$	([1L] on 3)
	5	$\Gamma;\Delta \vdash_{\curvearrowright} P :: \Lambda, x : \bot$	([↷] on 4)
• [⊥L]	1	$\Gamma;x : \bot \vdash_{\curvearrowright} x\langle\rangle.\mathbf{0} :: \emptyset$	(assumption)
	2	$\Gamma;\emptyset \vdash_{\curvearrowright} x\langle\rangle.\mathbf{0} :: x : \mathbf{1}$	([1R])
	3	$\Gamma;x : \bot \vdash_{\curvearrowright} x\langle\rangle.\mathbf{0} :: \emptyset$	([↶] on 2)
• [⅋R]	1	$\Gamma;\Delta \vdash_{\curvearrowright} x(y).P :: \Lambda, x : A \parr B$	(assumption)
	2	$\Gamma;\Delta \vdash_{\curvearrowright} P :: \Lambda, y : A, x : B$	(inversion on 1)

	3	$\Gamma; \Delta \vdash_? P :: \Lambda, y : A, x : B$ with only $*$ rules	(IH on 2)
	4	$\Gamma; \Delta, y : A^\perp, x : B^\perp \vdash_? P :: \Lambda$	($[\curvearrowleft]$ twice on 3)
	5	$\Gamma; \Delta, x : A^\perp \otimes B^\perp \vdash_? x(y).P :: \Lambda$	($[\otimes\text{L}]$ on 4)
	6	$\Gamma; \Delta \vdash_? x(y).P :: \Lambda, x : A \parr B$	($[\curvearrowright]$ on 5)
• $[\parr\text{L}]$	1	$\Gamma; \Delta, \Delta', x : A \parr B \vdash_? (\nu y)x\langle y\rangle.(P \mid Q) :: \Lambda, \Lambda'$	(assumption)
	2	$\Gamma; \Delta, y : A \vdash_? P :: \Lambda$	
	3	$\Gamma; \Delta', x : B \vdash_? Q :: \Lambda'$	(inversion on 1)
	4	$\Gamma; \Delta, y : A \vdash_? P :: \Lambda$ with only $*$ rules	(IH on 2)
	5	$\Gamma; \Delta', x : B \vdash_? Q :: \Lambda'$ with only $*$ rules	(IH on 3)
	6	$\Gamma; \Delta \vdash_? P :: \Lambda, y : A^\perp$	($[\curvearrowright]$ on 4)
	7	$\Gamma; \Delta' \vdash_? Q :: \Lambda', x : B^\perp$	($[\curvearrowright]$ on 5)
	8	$\Gamma; \Delta, \Delta' \vdash_? (\nu y)x\langle y\rangle.(P \mid Q) :: \Lambda, \Lambda', x : A^\perp \otimes B^\perp$	($[\otimes\text{R}]$ on 6 and 7)
	9	$\Gamma; \Delta, \Delta', x : A \parr B \vdash_? (\nu y)x\langle y\rangle.(P \mid Q) :: \Lambda, \Lambda'$	($[\curvearrowleft]$ on 8)
• [COPYR]	1	$\Gamma, u : A; \Delta \vdash_? (\nu x)u\langle x\rangle.P :: \Lambda$	(assumption)
	2	$\Gamma, u : A; \Delta \vdash_? P :: \Lambda, x : A^\perp$	(inversion on 1)
	3	$\Gamma, u : A; \Delta \vdash_? P :: \Lambda, x : A^\perp$ with only $*$ rules	(IH on 2)
	4	$\Gamma, u : A; \Delta, x : A \vdash_? P :: \Lambda$	($[\curvearrowleft]$ on 3)
	5	$\Gamma, u : A; \Delta \vdash_? (\nu x)u\langle x\rangle.P :: \Lambda$	([COPYL] on 4)
• [?R]	1	$\Gamma; \Delta \vdash_? P\{x/u\} :: \Lambda, x : ?A^\perp$	(assumption)
	2	$\Gamma, u : A; \Delta \vdash_? P :: \Lambda$	(inversion on 1)
	3	$\Gamma, u : A; \Delta \vdash_? P :: \Lambda$ with only $*$ rules	(IH on 2)
	4	$\Gamma; \Delta, x : !A \vdash_? P\{x/u\} :: \Lambda$	($[!\text{L}]$ on 3)
	5	$\Gamma; \Delta \vdash_? P\{x/u\} :: \Lambda, x : ?A^\perp$	($[\curvearrowright]$ on 4)
• [?L]	1	$\Gamma; x : ?A \vdash_? !x(y).P :: \emptyset$	(assumption)
	2	$\Gamma; y : A \vdash_? P :: \emptyset$	(inversion on 1)
	3	$\Gamma; y : A \vdash_? P :: \emptyset$ with only $*$ rules	(IH on 2)
	4	$\Gamma; \emptyset \vdash_? P :: y : A^\perp$	($[\curvearrowright]$ on 3)
	5	$\Gamma; \emptyset \vdash_? !x(y).P :: x : !A^\perp$	($[!\text{R}]$ on 4)
	6	$\Gamma; x : ?A \vdash_? !x(y).P :: \emptyset$	($[\curvearrowleft]$ on 5)
• [CUTRR]	1	$\Gamma; \Delta, \Delta' \vdash_? (\nu x)(P \mid Q) :: \Lambda, \Lambda'$	(assumption)
	2	$\Gamma; \Delta \vdash_? P :: \Lambda, x : A$	
	3	$\Gamma; \Delta' \vdash_? Q :: \Lambda', x : A^\perp$	(inversion on 1)
	4	$\Gamma; \Delta \vdash_? P :: \Lambda, x : A$ with only $*$ rules	(IH on 2)
	5	$\Gamma; \Delta' \vdash_? Q :: \Lambda', x : A^\perp$ with only $*$ rules	(IH on 3)
	6	$\Gamma; \Delta', x : A \vdash_? Q :: \Lambda'$	($[\curvearrowleft]$ on 5)
	7	$\Gamma; \Delta, \Delta' \vdash_? (\nu x)(P \mid Q) :: \Lambda, \Lambda'$	([CUTRL] on 4 and 6)
• [CUTLL]	1	$\Gamma; \Delta, \Delta' \vdash_? (\nu x)(P \mid Q) :: \Lambda, \Lambda'$	(assumption)
	2	$\Gamma; \Delta, x : A \vdash_? P :: \Lambda$	
	3	$\Gamma; \Delta', x : A^\perp \vdash_? Q :: \Lambda'$	(inversion on 1)
	4	$\Gamma; \Delta, x : A \vdash_? P :: \Lambda$ with only $*$ rules	(IH on 2)

	5	$\Gamma; \Delta', x : A^{\perp} \vdash_{?} Q :: \Lambda'$ with only $*$ rules	(IH on 3)
	6	$\Gamma; \Delta \vdash_{?} P :: \Lambda, x : A^{\perp}$	($[\curvearrowright]$ on 4)
	7	$\Gamma; \Delta, \Delta' \vdash_{?} (\nu x)(P \mid Q) :: \Lambda, \Lambda'$	([CUTRL] on 6 and 5)
• [CUT?R]	1	$\Gamma; \Delta \vdash_{?} (\nu u)(P \mid !u(x).Q) :: \Lambda$	(assumption)
	2	$\Gamma, u : A; \Delta \vdash_{?} P :: \Lambda$	
	3	$\Gamma; x : A^{\perp} \vdash_{?} Q :: \emptyset$	(inversion on 1)
	4	$\Gamma, u : A; \Delta \vdash_{?} P :: \Lambda$ with only $*$ rules	(IH on 2)
	5	$\Gamma; x : A^{\perp} \vdash_{?} Q :: \emptyset$ with only $*$ rules	(IH on 3)
	6	$\Gamma; \emptyset \vdash_{?} Q :: x : A$	($[\curvearrowright]$ on 5)
	7	$\Gamma; \Delta \vdash_{?} (\nu u)(P \mid !u(x).Q) :: \Lambda$	([CUT!R] on 4 and 6)
• [CUT?L]	1	$\Gamma; \Delta \vdash_{?} (\nu u)(!u(x).P \mid Q) :: \Lambda$	(assumption)
	2	$\Gamma; x : A^{\perp} \vdash_{?} P :: \emptyset$	
	3	$\Gamma, u : A; \Delta \vdash_{?} Q :: \Lambda$	(inversion on 1)
	4	$\Gamma; x : A^{\perp} \vdash_{?} P :: \emptyset$ with only $*$ rules	(IH on 2)
	5	$\Gamma, u : A; \Delta \vdash_{?} Q :: \Lambda$ with only $*$ rules	(IH on 3)
	6	$\Gamma; \emptyset \vdash_{?} P :: x : A$	($[\curvearrowright]$ on 4)
	7	$\Gamma; \Delta \vdash_{?} (\nu u)(!u(x).P \mid Q) :: \Lambda$	([CUT!L] on 6 and 5)

(Item 2) Suppose given a proof of $\Gamma; \Delta \vdash P :: \Lambda$, possibly with applications of Rules $[\curvearrowleft]$ and $[\curvearrowright]$. By applying induction on the structure of this proof we show that it can be transformed to not contain any applications of $[\curvearrowleft]$ and $[\curvearrowright]$. We discuss every possible last-applied rule. However, all cases except $[\curvearrowleft]$ and $[\curvearrowright]$ follow directly from the induction hypothesis. Therefore, we only detail the case of $[\curvearrowleft]$—the case of $[\curvearrowright]$ is analogous.

Since $[\curvearrowleft]$ is the last-applied rule, we know the assumption is of the form $\Gamma; \Delta, x : A^{\perp} \vdash P :: \Lambda$. By inversion, we have $\Gamma; \Delta \vdash P :: \Lambda, x : A$. The idea is to move the application of $[\curvearrowleft]$ up the proof tree, applied to a subterm of A. We apply the induction hypothesis to find a proof of $\Gamma; \Delta \vdash P :: \Lambda, x : A$ without applications of $[\curvearrowleft]$ and $[\curvearrowright]$. Typing rules leave all channels/types untouched except the ones they work on. Therefore, we can traverse up the proof tree—remembering which steps were taken—until we encounter the rule that introduces $x : A$. Note that these steps do not include applications of $[\curvearrowleft]$ and $[\curvearrowright]$. The consequence of this rule looks like $\Gamma'; \Delta' \vdash P' :: \Lambda', x : A$, for some Γ', Δ', P' and Λ'. Now, we apply induction on the size of A (with induction hypothesis denoted IH_2) to prove $\Gamma'; \Delta', x : A^{\perp} \vdash P' :: \Lambda'$. We discuss every possible last-applied rule that introduces $x : A$:

• [IDR]	1	$\Gamma'; y : A \vdash [y \leftrightarrow x] :: x : A$	(assumption)
	2	$\Gamma'; y : A, x : A^{\perp} \vdash [y \leftrightarrow x] :: \emptyset$	([IDL])
• [$\mathbf{1}$R]	1	$\Gamma'; \emptyset \vdash x\langle\rangle.\mathbf{0} :: x : \mathbf{1}$	(assumption)
	2	$\Gamma'; x : \bot \vdash x\langle\rangle.\mathbf{0} :: \emptyset$	([$\bot$L])
• [$\bot$R]	1	$\Gamma'; \Delta' \vdash x().P' :: \Lambda', x : \bot$ without $[\curvearrowleft]/[\curvearrowright]$	(assumption)
	2	$\Gamma'; \Delta' \vdash P' :: \Lambda'$	(inversion on 1)
	3	$\Gamma'; \Delta', x : \mathbf{1} \vdash x().P' :: \Lambda'$	([$\mathbf{1}$L] on 2)
• [$\otimes$R]	1	$\Gamma'; \Delta', \Delta'' \vdash (\nu y)x\langle y\rangle.(P' \mid Q') :: \Lambda', \Lambda'', x : B \otimes C$ without $[\curvearrowleft]/[\curvearrowright]$	(assumption)
	2	$\Gamma'; \Delta' \vdash P' :: \Lambda', y : B$	
	3	$\Gamma'; \Delta'' \vdash Q' :: \Lambda'', x : C$	(inversion on 1)
	4	$\Gamma'; \Delta', y : B^{\perp} \vdash P' :: \Lambda'$	($[\curvearrowleft]$ on 2)
	5	$\Gamma'; \Delta'', x : C^{\perp} \vdash Q' :: \Lambda''$	($[\curvearrowleft]$ on 3)
	6	$\Gamma'; \Delta', y : B^{\perp} \vdash P' :: \Lambda'$ without $[\curvearrowleft]/[\curvearrowright]$	(IH_2 on 4)
	7	$\Gamma'; \Delta'', x : C^{\perp} \vdash Q' :: \Lambda''$ without $[\curvearrowleft]/[\curvearrowright]$	(IH_2 on 5)

	8	$\Gamma'; \Delta', \Delta'', x : B^\perp \parr C^\perp \vdash (\nu y)x\langle y\rangle.(P' \mid Q') :: \Lambda', \Lambda''$	([$\parr$L] on 8 and 9)
• [$\multimap$R]	1	$\Gamma'; \Delta' \vdash x(y).P' :: \Lambda', x : B \multimap C$ without [$\curvearrowleft$]/[$\curvearrowright$]	(assumption)
	2	$\Gamma'; \Delta', y : B \vdash P' :: \Lambda', x : C$	(inversion on 1)
	3	$\Gamma'; \Delta', y : B, x : C^\perp \vdash P' :: \Lambda'$	([$\curvearrowleft$] on 2)
	4	$\Gamma'; \Delta', y : B, x : C^\perp \vdash P' :: \Lambda'$ without [$\curvearrowleft$]/[$\curvearrowright$]	(IH_2 on 3)
	5	$\Gamma'; \Delta', x : B \otimes C^\perp \vdash x(y).P' :; \Lambda'$	([$\otimes$L] on 4)
• [$\oplus$R]	1	$\Gamma'; \Delta' \vdash x \lhd j.P' :: \Lambda', x : \oplus\{i : A_i\}_{i\in I}$ without [$\curvearrowleft$]/[$\curvearrowright$]	(assumption)
	2	$\Gamma'; \Delta' \vdash P' :: \Lambda', x : A_j$	
	3	$j \in I$	(inversion on 1)
	4	$\Gamma'; \Delta', x : A_j^\perp \vdash P' :: \Lambda'$	([$\curvearrowleft$] on 3)
	5	$\Gamma'; \Delta', x : A_j^\perp \vdash P' :: \Lambda'$	(IH_2 on 4)
	6	$\Gamma'; \Delta', x : \&\{i : A_i^\perp\}_{i\in I} \vdash x \lhd j.P' :: \Lambda'$	([&L] on 5 and 3)
• [&R]	1	$\Gamma'; \Delta' \vdash x \rhd \{i : P_i'\}_{i\in I} :: \Lambda', x : \&\{i : A_i\}_{i\in I}$ without [$\curvearrowleft$]/[$\curvearrowright$]	(assumption)
	2	$\forall i \in I.\ \Gamma'; \Delta' \vdash P_i' :: \Lambda', x : A_i$	(inversion on 1)
	3	$\forall i \in I.\ \Gamma'; \Delta', x : A_i^\perp \vdash P_i' :: \Lambda'$	([$\curvearrowleft$] on 2)
	4	$\forall i \in I.\ \Gamma'; \Delta', x : A_i^\perp \vdash P_i' :: \Lambda'$ without [$\curvearrowleft$]/[$\curvearrowright$]	(IH_2 on 3)
	5	$\Gamma'; \Delta', x : \oplus\{i : A_i^\perp\}_{i\in I} \vdash x \rhd \{i : P_i'\}_{i\in I} :: \Lambda'$	([$\oplus$L] on 4)
• [!R]	1	$\Gamma'; \emptyset \vdash !x(y).P' :: x : !B$ without [$\curvearrowleft$]/[$\curvearrowright$]	(assumption)
	2	$\Gamma'; \emptyset \vdash P' :: y : B$	(inversion on 1)
	3	$\Gamma'; y : B^\perp \vdash P' :: \emptyset$	([$\curvearrowleft$] on 2)
	4	$\Gamma'; y : B^\perp \vdash P' :: \emptyset$ without [$\curvearrowleft$]/[$\curvearrowright$]	(IH_2 on 3)
	5	$\Gamma'; x : ?B^\perp \vdash !x(y).P' :: \emptyset$	([?L] on 4)
• [?R]	1	$\Gamma'; \Delta' \vdash P'\{x/u\} :: \Lambda', x : ?B$	(assumption)
	2	$\Gamma', u : A^\perp; \Delta' \vdash P' :: \Lambda'$	(inversion on 1)
	3	$\Gamma'; \Delta', x : !B^\perp \vdash P'\{x/u\} :: \Lambda'$	([!L] on 2)

Finally, we recall the steps we have traversed up the tree and remember that they do not include applications of [$\curvearrowleft$] and [$\curvearrowright$]. We re-apply them on $\Gamma'; \Delta', x : A^\perp \vdash P' :: \Lambda'$, without affecting $x : A^\perp$. Instead, they only affect Γ', Δ' and Λ' to give us a proof of $\Gamma; \Delta, x : A^\perp \vdash P :: \Lambda$ without applications of [$\curvearrowleft$] and [$\curvearrowright$].

This concludes the proof of Theorem 2.5. □

3. Comparing intuitionistic and classical interpretations

In this section, we rigorously compare the class of πULL-typable processes to the classes of processes typable in session type interpretations of linear logic, in classical [12,56] and intuitionistic [7] settings. πULL is an independent yardstick for this comparison, because it is derived from ULL which subsumes both linear logics by design.

We have discussed in Section 2 several design choices we made when defining ULL and πULL. Besides differences stemming from the dichotomy between classical and intuitionistic linear logic, logically-motivated session type systems also present features induced by certain design choices. For a fair comparison, we want to make sure that the differences come only from typing. This means that we need to make the same design choices for both interpretations: we require explicit closing, a separate unrestricted context, and identity as forwarding.

3.1. Session type systems derived from intuitionistic and classical linear logic

Our goal is to derive session type systems from intuitionistic and classical linear logic for the same process syntax as πULL uses (cf. Definition 2.3).

$$[\text{ID}]\ \frac{}{\Gamma; x : A \vdash_I [x \leftrightarrow y] :: y : A} \qquad [\mathbf{1}\text{R}]\ \frac{}{\Gamma; \emptyset \vdash_I x\langle\rangle.\mathbf{0} :: x : \mathbf{1}} \qquad [\mathbf{1}\text{L}]\ \frac{\Gamma; \Delta \vdash_I P :: z : C}{\Gamma; \Delta, x : \mathbf{1} \vdash_I x().P :: z : C}$$

$$[\otimes\text{R}]\ \frac{\Gamma; \Delta \vdash_I P :: y : A \qquad \Gamma; \Delta' \vdash_I Q :: x : B}{\Gamma; \Delta, \Delta' \vdash_I (\nu y)x\langle y\rangle.(P \mid Q) :: x : A \otimes B} \qquad [\otimes\text{L}]\ \frac{\Gamma; \Delta, y : A, x : B \vdash_I P :: z : C}{\Gamma; \Delta, x : A \otimes B \vdash_I x(y).P :: z : C}$$

$$[\multimap\text{R}]\ \frac{\Gamma; \Delta, y : A \vdash_I P :: x : B}{\Gamma; \Delta \vdash_I x(y).P :: x : A \multimap B} \qquad [\multimap\text{L}]\ \frac{\Gamma; \Delta \vdash_I P :: y : A \qquad \Gamma; \Delta', x : B \vdash_I Q :: z : C}{\Gamma; \Delta, \Delta', x : A \multimap B \vdash_I (\nu y)x\langle y\rangle.(P \mid Q) :: z : C}$$

$$[\oplus\text{R}]\ \frac{\Gamma; \Delta \vdash_I P :: x : A_j \qquad j \in I}{\Gamma; \Delta \vdash_I x \lhd j.P :: x : \oplus\{i : A_i\}_{i \in I}} \qquad [\oplus\text{L}]\ \frac{\forall i \in I.\ \Gamma; \Delta, x : A_i \vdash_I P_i :: z : C}{\Gamma; \Delta, x : \oplus\{i : A_i\}_{i \in I} \vdash_I x \rhd \{i : P_i\}_{i \in I} :: z : C}$$

$$[\&\text{R}]\ \frac{\forall i \in I.\ \Gamma; \Delta \vdash_I P_i :: x : A_i}{\Gamma; \Delta \vdash_I x \rhd \{i : P_i\}_{i \in I} :: x : \&\{i : A_i\}_{i \in I}} \qquad [\&\text{L}]\ \frac{\Gamma; \Delta, x : A_j \vdash_I P :: z : C \qquad j \in I}{\Gamma; \Delta, x : \&\{i : A_i\}_{i \in I} \vdash_I x \lhd j.P :: z : C}$$

$$[\text{COPY}]\ \frac{\Gamma, u : A; \Delta, x : A \vdash_I P :: z : C}{\Gamma, u : A; \Delta \vdash_I (\nu x)u\langle x\rangle.P :: z : C} \qquad [!\text{R}]\ \frac{\Gamma; \emptyset \vdash_I P :: y : A}{\Gamma; \emptyset \vdash_I\ !x(y).P :: x : !A} \qquad [!\text{L}]\ \frac{\Gamma, u : A; \Delta \vdash_I P :: z : C}{\Gamma; \Delta, x : !A \vdash_I P\{x/u\} :: z : C}$$

$$[\text{CUTRL}]\ \frac{\Gamma; \Delta \vdash_I P :: x : A \qquad \Gamma; \Delta', x : A \vdash_I Q :: z : C}{\Gamma; \Delta, \Delta' \vdash_I (\nu x)(P \mid Q) :: z : C} \qquad [\text{CUTLR}]\ \frac{\Gamma; \Delta, x : A \vdash_I P :: z : C \qquad \Gamma; \Delta' \vdash_I Q :: x : A}{\Gamma; \Delta, \Delta' \vdash_I (\nu x)(P \mid Q) :: z : C}$$

$$[\text{CUT!R}]\ \frac{\Gamma, u : A; \Delta \vdash_I P :: z : C \qquad \Gamma; \emptyset \vdash_I Q :: x : A}{\Gamma; \Delta \vdash_I (\nu u)(P \mid !u(x).Q) :: z : C} \qquad [\text{CUT!L}]\ \frac{\Gamma; \emptyset \vdash_I P :: x : A \qquad \Gamma, u : A; \Delta \vdash_I Q :: z : C}{\Gamma; \Delta \vdash_I (\nu u)(!u(x).P \mid Q) :: z : C}$$

Fig. 4. The πILL type system.

The intuitionistic type system (πILL) Fig. 4 gives the inference rules for the type system derived from *intuitionistic* linear logic, denoted πILL. Following the presentation by Caires, Pfenning, and Toninho in [11], propositions/types for the intuitionistic type system are generated by the following grammar (a fragment of Definition 2.1):

$$A, B ::= \mathbf{1} \mid A \otimes B \mid A \multimap B \mid \oplus\{i : A\}_{i \in I} \mid \&\{i : A\}_{i \in I} \mid !A$$

Notice how, following most presentations of intuitionistic linear logic (e.g., [25,3,38]), πILL propositions do not include $\bot$ and ?. The judgment is denoted as follows:

$$\Gamma; \Delta \vdash_I P :: z : C$$

With respect to the intuitionistic interpretation introduced in [7,12], πILL features a non-silent interpretation of $\mathbf{1}$, based on explicit closure of sessions (cf. Rules [$\mathbf{1}$R] and [$\mathbf{1}$L]). We adopt this interpretation because, as explained in [11], it leads to a Curry-Howard correspondence that is tighter than correspondences with silent interpretations (such as those in [7,12]).

A more superficial difference is that πILL follows standard presentations of session type systems by supporting n-ary labeled choices; in contrast, the systems in [7,12] support binary labeled choices. We also include symmetric variants of the cut-rules, as they are necessary for type preservation under Rule [CutSymm] of structural congruence. As expected, none of these cut-rules make use of the duality relation in Definition 2.2.

The classical type system (πCLL*) Fig. 5 gives the inference rules for the type system derived from *classical* linear logic. It is based on a combination of features from Caires, Pfenning, and Toninho's πCLL in [12] and Wadler's CP in [56]; in the following, it is denoted πCLL*. The syntax of propositions/types is exactly as in Definition 2.1. The corresponding judgment is as follows:

$$P \vdash_C \Gamma; \Delta$$

Table 2 summarizes the differences in the design choices between πCLL, CP, and πCLL*; these differences are merely superficial:

- As we have seen, an explicit closing of sessions (as in CP) concerns a non-silent interpretation of the atomic propositions $\mathbf{1}$ and $\bot$. In contrast, πCLL realizes an implicit (silent) closing of sessions.
- Sequents with a separate unrestricted context (as in πCLL) are of the form $P \vdash \Gamma; \Delta$, which can also be written as $P \vdash \Delta, \Gamma'$ where Γ' contains only types of the form $!A$.
- The identity axiom can be interpreted as the forwarding process, which enables CP to account for behavioral polymorphism (i.e., universal and existential quantification over propositions/session types) [56,9]. As already mentioned, forwarding is not a typical process construct in session π-calculi.

$$[\text{ID}]\ \frac{}{[x \leftrightarrow y] \vdash_C \Gamma; x : A, y : A^\perp} \qquad [\bot]\ \frac{P \vdash_C \Gamma; \Delta}{x().P \vdash_C \Gamma; \Delta, x : \bot} \qquad [\mathbf{1}]\ \frac{}{x\langle\rangle.\mathbf{0} \vdash_C \Gamma; x : \mathbf{1}}$$

$$[\otimes]\ \frac{P \vdash_C \Gamma; \Delta, y : A \qquad Q \vdash_C \Gamma; \Delta', x : B}{(\nu y)x\langle y\rangle.(P \mid Q) \vdash_C \Gamma; \Delta, \Delta', x : A \otimes B} \qquad [\parr]\ \frac{P \vdash_C \Gamma; \Delta, y : A, x : B}{x(y).P \vdash_C \Gamma; \Delta, x : A \parr B}$$

$$[\oplus]\ \frac{P \vdash_C \Gamma; \Delta, x : A_j \qquad j \in I}{x \lhd j.P \vdash_C \Gamma; \Delta, x : \oplus\{i : A_i\}_{i \in I}} \qquad [\&]\ \frac{\forall i \in I.\ P_i \vdash_C \Gamma; \Delta, x : A_i}{x \rhd \{i : P_i\}_{i \in I} \vdash_C \Gamma; \Delta, x : \&\{i : A_i\}_{i \in I}}$$

$$[\text{COPY}]\ \frac{P \vdash_C \Gamma, u : A; \Delta, y : A}{(\nu y)u\langle y\rangle.P \vdash_C \Gamma, u : A; \Delta} \qquad [?]\ \frac{P \vdash_C \Gamma, u : A; \Delta}{P\{x/u\} \vdash_C \Gamma; \Delta, x : ?A}$$

$$[!]\ \frac{P \vdash_C \Gamma; y : A}{!x(y).P \vdash_C \Gamma; x : !A} \qquad [\text{CUT}]\ \frac{P \vdash_C \Gamma; \Delta, x : A \qquad Q \vdash_C \Gamma; \Delta', x : A^\perp}{(\nu x)(P \mid Q) \vdash_C \Gamma; \Delta, \Delta'}$$

$$[\text{CUT?R}]\ \frac{P \vdash_C \Gamma, u : A; \Delta \qquad Q \vdash_C \Gamma; x : A^\perp}{(\nu u)(P \mid !u(x).Q) \vdash_C \Gamma; \Delta} \qquad [\text{CUT?L}]\ \frac{P \vdash_C \Gamma; x : A^\perp \qquad Q \vdash_C \Gamma, u : A; \Delta}{(\nu u)(!u(x).P \mid Q) \vdash_C \Gamma; \Delta}$$

Fig. 5. The $\pi\mathsf{CLL}^\star$ type system.

Table 2
Feature comparison of three session type interpretations of classical linear logic.

	Explicit closing	Separate unrestricted context	Identity as forwarding
πCLL [12]	No	Yes	No
CP [56]	Yes	No	Yes
$\pi\mathsf{CLL}^\star$ (this paper)	Yes	Yes	Yes

Note that, since $\pi\mathsf{CLL}^\star$ typing judgments are one-sided, there is no need for symmetric cut-rules. The three cut-rules in Fig. 5 use duality as in Definition 2.2.

3.2. Formal comparison

Now that we have presented all three systems, we start our comparison by contrasting the shape of their typing judgments:

$$\Gamma; \Delta \vdash P :: \Lambda \qquad \Gamma; \Delta \vdash_{\mathcal{I}} P :: x : A \qquad P \vdash_C \Gamma; \Delta$$

In πULL and πILL judgments are similar, but in πILL they have exactly one channel/type pair on the right. We will see that the difference between πULL and πILL can be characterized by this fact alone. Judgments in $\pi\mathsf{CLL}^\star$ are different from those in πULL and πILL: they have only one linear context and both the linear and the unrestricted contexts are on the right. As we will see, our results reflect this with a duality relation between the contexts of πULL and $\pi\mathsf{CLL}^\star$.

Our formal results rely on classes of processes typable in the three typing systems:

Definition 3.1. Let $\mathbb{P}$ denote the set of all processes induced by Definition 2.3. Then

$$\mathcal{U} = \{P \in \mathbb{P} \mid \exists \Gamma, \Delta, \Lambda \text{ such that } \Gamma; \Delta \vdash P :: \Lambda\},$$
$$\mathcal{C} = \{P \in \mathbb{P} \mid \exists \Gamma, \Delta \text{ such that } P \vdash_C \Gamma; \Delta\},$$
$$\mathcal{I} = \{P \in \mathbb{P} \mid \exists \Gamma, \Delta, x, A \text{ such that } \Gamma; \Delta \vdash_{\mathcal{I}} P :: x : A\}.$$

Our first result is that $\mathcal{U} = \mathcal{C}$, i.e., πULL is merely a two-sided representation of $\pi\mathsf{CLL}^\star$.

Theorem 3.1. $\mathcal{U} = \mathcal{C}$.

We briefly discuss how we prove this result. On the one hand (from left to right), if $P \in \mathcal{U}$, there is a proof of $\Gamma; \Delta \vdash P :: \Lambda$. By backtracking on this proof, we can use rules in $\pi\mathsf{CLL}^\star$ analogous to the πULL-rules used to generate an equivalent proof of $P \vdash_C (\Gamma)^\perp; (\Delta)^\perp, \Lambda$ thus showing $P \in \mathcal{C}$. Note how the single-sidedness of $\pi\mathsf{CLL}^\star$ judgments requires us to move Γ and Δ to the right-hand side using duality.

On the other hand (from right to left), if $P \in \mathcal{C}$, there is a proof of $P \vdash_{\mathcal{C}} \Gamma; \Delta$. Again, by backtracking on this proof, we can use a combination of rules similar to those used in $\pi\mathsf{CLL}^{\star}$ in combination with Rules $[\curvearrowleft]$ and $[\curvearrowright]$ from Definition 2.6 to prove $(\Gamma)^{\perp}; \emptyset \vdash_{\curvearrowright} \Delta$ in $\pi\mathsf{ULL}_{\curvearrowright}$. Going through $\pi\mathsf{ULL}_{\curvearrowright}$ simplifies this process, since using $[\curvearrowleft]$ and $[\curvearrowright]$ enables us to guarantee that all of Δ ends up on the right-hand side of the typing judgment instead of dividing unpredictably between left and right. Note that we do have to use duality to move Γ to the left. Since $[\curvearrowleft]$ and $[\curvearrowright]$ are admissible in $\pi\mathsf{ULL}$ (cf. Theorem 2.5), this means we also have a proof of $(\Gamma)^{\perp}; \emptyset \vdash \Delta$ in $\pi\mathsf{ULL}$. Hence, $P \in \mathcal{U}$.

Proof of Theorem 3.1. *($\mathcal{U} \subseteq \mathcal{C}$)* Take any $P \in \mathcal{U}$. Then, by Definition 3.1, there are Γ, Δ, Λ s.t. $\Gamma; \Delta \vdash P :: \Lambda$. By showing that this implies $P \vdash_{\mathcal{C}} (\Gamma)^{\perp}; (\Delta)^{\perp}, \Lambda$, we have $P \in \mathcal{C}$. We show this by induction on the structure of the proof of $\Gamma; \Delta \vdash P :: \Lambda$.

• [IDR]	1	$\Gamma; x : A \vdash [x \leftrightarrow y] :: y : A$	(assumption)
	2	$[x \leftrightarrow y] \vdash_{\mathcal{C}} (\Gamma)^{\perp}; x : A^{\perp}, y : A$	([ID])
• [IDL]	1	$\Gamma; x : A, y : A^{\perp} \vdash [x \leftrightarrow y] :: \emptyset$	(assumption)
	2	$[x \leftrightarrow y] \vdash_{\mathcal{C}} (\Gamma)^{\perp}; x : A^{\perp}, y : A$	([ID])
• [**1**R]	1	$\Gamma; \emptyset \vdash x\langle\rangle.\mathbf{0} :: x : \mathbf{1}$	(assumption)
	2	$x\langle\rangle.\mathbf{0} \vdash_{\mathcal{C}} (\Gamma)^{\perp}; x : \mathbf{1}$	([**1**])
• [**1**L]	1	$\Gamma; \Delta, x : \mathbf{1} \vdash x().P :: \Lambda$	(assumption)
	2	$\Gamma; \Delta \vdash P :: \Lambda$	(inversion on 1)
	3	$P \vdash_{\mathcal{C}} (\Gamma)^{\perp}; (\Delta)^{\perp}, \Lambda$	(IH on 2)
	4	$x().P \vdash_{\mathcal{C}} (\Gamma)^{\perp}; (\Delta)^{\perp}, \Lambda, x : \perp$	([$\perp$] on 3)
• [$\perp$R]	1	$\Gamma; \Delta \vdash x().P :: \Lambda, x : \perp$	(assumption)
	2	$\Gamma; \Delta \vdash P :: \Lambda$	(inversion on 1)
	3	$P \vdash_{\mathcal{C}} (\Gamma)^{\perp}; (\Delta)^{\perp}, \Lambda$	(IH on 2)
	4	$x().P \vdash_{\mathcal{C}} (\Gamma)^{\perp}; (\Delta)^{\perp}, \Lambda, x : \perp$	([$\perp$] on 3)
• [$\perp$L]	1	$\Gamma; x : \perp \vdash x\langle\rangle.\mathbf{0} :: \emptyset$	(assumption)
	2	$x\langle\rangle.\mathbf{0} \vdash_{\mathcal{C}} (\Gamma)^{\perp}; x : \mathbf{1}$	([**1**])
• [$\otimes$R]	1	$\Gamma; \Delta, \Delta' \vdash (\nu y)x\langle y\rangle.(P \mid Q) :: \Lambda, \Lambda', x : A \otimes B$	(assumption)
	2	$\Gamma; \Delta \vdash P :: \Lambda, y : A$	
	3	$\Gamma; \Delta' \vdash Q :: \Lambda', x : B$	(inversion on 1)
	4	$P \vdash_{\mathcal{C}} (\Gamma)^{\perp}; (\Delta)^{\perp}, \Lambda, y : A$	(IH on 2)
	5	$Q \vdash_{\mathcal{C}} (\Gamma)^{\perp}; (\Delta')^{\perp}, \Lambda', x : B$	(IH on 3)
	6	$(\nu y)x\langle y\rangle.(P \mid Q) \vdash_{\mathcal{C}} (\Gamma)^{\perp}; (\Delta)^{\perp}, (\Delta')^{\perp}, \Lambda, \Lambda', x : A \otimes B$	([$\otimes$] on 4 and 5)
• [$\otimes$L]	1	$\Gamma; \Delta, x : A \otimes B \vdash x(y).P :: \Lambda$	(assumption)
	2	$\Gamma; \Delta, y : A, x : B \vdash P :: \Lambda$	(inversion on 1)
	3	$P \vdash_{\mathcal{C}} (\Gamma)^{\perp}; (\Delta)^{\perp}, \Lambda, y : A^{\perp}, x : B^{\perp}$	(IH on 2)
	4	$x(y).P \vdash_{\mathcal{C}} (\Gamma)^{\perp}; (\Delta)^{\perp}, \Lambda, x : A^{\perp} \parr B^{\perp}$	([$\parr$] on 3)
• [$\parr$R]	1	$\Gamma; \Delta \vdash x(y).P :: \Lambda, x : A \parr B$	(assumption)
	2	$\Gamma; \Delta \vdash P :: \Lambda, y : A, x : B$	(inversion on 1)
	3	$P \vdash_{\mathcal{C}} (\Gamma)^{\perp}; (\Delta)^{\perp}, \Lambda, y : A, x : B$	(IH on 2)
	4	$x(y).P \vdash_{\mathcal{C}} (\Gamma)^{\perp}; (\Delta)^{\perp}, \Lambda, x : A \parr B$	([$\parr$] on 3)
• [$\parr$L]	1	$\Gamma; \Delta, \Delta', x : A \parr B \vdash (\nu y)x\langle y\rangle.(P \mid Q) :: \Lambda, \Lambda'$	(assumption)
	2	$\Gamma; \Delta, y : A \vdash P :: \Lambda$	
	3	$\Gamma; \Delta', x : B \vdash Q :: \Lambda'$	(inversion on 1)

	4	$P \vdash_C (\Gamma)^\perp; (\Delta)^\perp, \Lambda, y : A^\perp$	(IH on 2)
	5	$Q \vdash_C (\Gamma)^\perp; (\Delta')^\perp, \Lambda', x : B^\perp$	(IH on 3)
	6	$(\nu y)x\langle y\rangle.(P \mid Q) \vdash_C (\Gamma)^\perp; (\Delta)^\perp, (\Delta')^\perp, \Lambda, \Lambda', x : A^\perp \otimes B^\perp$	([⊗] on 4 and 5)
• [⊸R]	1	$\Gamma; \Delta \vdash x(y).P :: \Lambda, x : A \multimap B$	(assumption)
	2	$\Gamma; \Delta, y : A \vdash P :: \Lambda, x : B$	(inversion on 1)
	3	$P \vdash_C (\Gamma)^\perp; (\Delta)^\perp, \Lambda, y : A^\perp, x : B$	(IH on 2)
	4	$x(y).P \vdash_C (\Gamma)^\perp; (\Delta)^\perp, \Lambda, x : A^\perp \parr B$	([⅋] on 3)
• [⊸L]	1	$\Gamma; \Delta, \Delta', x : A \multimap B \vdash (\nu y)x\langle y\rangle.(P \mid Q) :: \Lambda, \Lambda'$	(assumption)
	2	$\Gamma; \Delta \vdash P :: \Lambda, y : A$	
	3	$\Gamma; \Delta', x : B \vdash Q :: \Lambda'$	(inversion on 1)
	4	$P \vdash_C (\Gamma)^\perp; (\Delta)^\perp, \Lambda, y : A$	(IH on 2)
	5	$Q \vdash_C (\Gamma)^\perp; (\Delta')^\perp, \Lambda', x : B^\perp$	(IH on 3)
	6	$(\nu y)x\langle y\rangle.(P \mid Q) \vdash_C (\Gamma)^\perp; (\Delta)^\perp, (\Delta')^\perp, \Lambda, \Lambda', x : A \otimes B^\perp$	([⊗] on 4 and 5)
• [&R]	1	$\Gamma; \Delta \vdash x \triangleright \{i : P_i\}_{i\in I} :: \Lambda, x : \&\{i : A_i\}_{i\in I}$	(assumption)
	2	$\forall i \in I.\ \Gamma; \Delta \vdash P_i :: \Lambda, x : A_i$	(inversion on 1)
	3	$\forall i \in I.\ P_i \vdash_C (\Gamma)^\perp; (\Delta)^\perp, \Lambda, x : A_i$	(IH on 2)
	4	$x \triangleright \{i : P_i\}_{i\in I} \vdash_C (\Gamma)^\perp; (\Delta)^\perp, \Lambda, x : \&\{i : A_i\}_{i\in I}$	([&] on 3)
• [&L]	1	$\Gamma; \Delta, x : \&\{i : A_i\} \vdash x \triangleleft j.P :: \Lambda$	(assumption)
	2	$\Gamma; \Delta, x : A_j \vdash P :: \Lambda$	
	3	$j \in I$	(inversion on 1)
	4	$P \vdash_C (\Gamma)^\perp; (\Delta)^\perp, \Lambda, x : A_j^\perp$	(IH on 2)
	5	$x \triangleright j.P \vdash_C (\Gamma)^\perp; (\Delta)^\perp, \Lambda, x : \oplus\{i : A_i^\perp\}_{i\in I}$	([⊕] on 4 and 3)
• [⊕R]	1	$\Gamma; \Delta \vdash x \triangleleft j.P :: \Lambda, x : \oplus\{i : A_i\}_{i\in I}$	(assumption)
	2	$\Gamma; \Delta \vdash P :: \Lambda, x : A_j$	
	3	$j \in I$	(inversion on 1)
	4	$P \vdash_C (\Gamma)^\perp; (\Delta)^\perp, \Lambda, x : A_j$	(IH on 2)
	5	$x \triangleleft j.P \vdash_C (\Gamma)^\perp; (\Delta)^\perp, \Lambda, x : \oplus\{i : A_i\}_{i\in I}$	([⊕] on 4 and 3)
• [⊕L]	1	$\Gamma; \Delta, x : \oplus\{i : A_i\}_{i\in I} \vdash x \triangleright \{i : P_i\}_{i\in I} :: \Lambda$	(assumption)
	2	$\forall i \in I.\ \Gamma; \Delta, x : A_i \vdash P_i :: \Lambda$	(inversion on 1)
	3	$\forall i \in I.\ P_i \vdash_C (\Gamma)^\perp; (\Delta)^\perp, \Lambda, x : A_i^\perp$	(IH on 2)
	4	$x \triangleright \{i : P_i\}_{i\in I} \vdash_C (\Gamma)^\perp; (\Delta)^\perp, \Lambda, x : \&\{i : A_i^\perp\}_{i\in I}$	([&] on 4)
• [copyR]	1	$\Gamma, u : A; \Delta \vdash (\nu x)u\langle x\rangle.P :: \Lambda$	(assumption)
	2	$\Gamma, u : A; \Delta \vdash P :: \Lambda, x : A^\perp$	(inversion on 1)
	3	$P \vdash_C (\Gamma)^\perp, u : A^\perp; (\Delta)^\perp, \Lambda, x : A^\perp$	(IH on 2)
	4	$(\nu x)u\langle x\rangle.P \vdash_C (\Gamma)^\perp, u : A^\perp; (\Delta^\perp), \Lambda$	([copy] on 3)
• [copyL]	1	$\Gamma, u : A; \Delta \vdash (\nu x)u\langle x\rangle.P :: \Lambda$	(assumption)
	2	$\Gamma, u : A; \Delta, x : A \vdash P :: \Lambda$	(inversion on 1)
	3	$P \vdash_C (\Gamma)^\perp, u : A^\perp; (\Delta)^\perp, \Lambda, x : A^\perp$	(IH on 2)
	4	$(\nu x)u\langle x\rangle.P :: (\Gamma)^\perp, u : A^\perp; (\Delta)^\perp, \Lambda$	([copy] on 3)

• [!R]	1	$\Gamma; \emptyset \vdash !x(y).P :: x : !A$	(assumption)
	2	$\Gamma; \emptyset \vdash P :: y : A$	(inversion on 1)
	3	$P \vdash_C (\Gamma)^\perp; y : A$	(IH on 2)
	4	$!x(y).P \vdash_C (\Gamma)^\perp; y : !A$	([!] on 3)
• [!L]	1	$\Gamma; \Delta, x : !A \vdash P\{x/u\} :: \Lambda$	(assumption)
	2	$\Gamma, u : A; \Delta \vdash P :: \Lambda$	(inversion on 1)
	3	$P \vdash_C (\Gamma)^\perp, u : A^\perp; (\Delta)^\perp, \Lambda$	(IH on 2)
	4	$P\{x/u\} \vdash_C (\Gamma)^\perp; (\Delta)^\perp, \Lambda, x : ?A^\perp$	([?] on 3)
• [?R]	1	$\Gamma; \Delta \vdash P\{x/u\} :: \Lambda, x : ?A^\perp$	(assumption)
	2	$\Gamma, u : A; \Delta \vdash P :: \Lambda$	(inversion on 1)
	3	$P \vdash_C (\Gamma)^\perp, u : A^\perp; (\Delta)^\perp, \Lambda$	(IH on 2)
	4	$P\{x/u\} \vdash_C (\Gamma)^\perp; (\Delta)^\perp, \Lambda, x : ?A^\perp$	([?] on 3)
• [?L]	1	$\Gamma; x : ?A \vdash !x(y).P :: \emptyset$	(assumption)
	2	$\Gamma; y : A \vdash P :: \emptyset$	(inversion on 1)
	3	$P \vdash_C (\Gamma)^\perp; y : A^\perp$	(IH on 2)
	4	$!x(y).P \vdash_C (\Gamma)^\perp; x : !A^\perp$	([!] on 3)
• [CUTRL]	1	$\Gamma; \Delta, \Delta' \vdash (\nu x)(P \mid Q) :: \Lambda, \Lambda'$	(assumption)
	2	$\Gamma; \Delta \vdash P :: \Lambda, x : A$	
	3	$\Gamma; \Delta', x : A \vdash Q :: \Lambda'$	(inversion on 1)
	4	$P \vdash_C (\Gamma)^\perp; (\Delta)^\perp, \Lambda, x : A$	(IH on 2)
	5	$Q \vdash_C (\Gamma)^\perp; (\Delta')^\perp, \Lambda', x : A^\perp$	(IH on 3)
	6	$(\nu x)(P \mid Q) \vdash_C (\Gamma)^\perp; (\Delta)^\perp, (\Delta')^\perp, \Lambda, \Lambda'$	([CUT] on 4 and 5)
• [CUTLR]	1	$\Gamma; \Delta, \Delta' \vdash (\nu x)(P \mid Q) :: \Lambda, \Lambda'$	(assumption)
	2	$\Gamma; \Delta, x : A \vdash P :: \Lambda$	
	3	$\Gamma; \Delta' \vdash Q :: \Lambda', x : A$	(inversion on 1)
	4	$P \vdash_C (\Gamma)^\perp; (\Delta)^\perp, \Lambda, x : A^\perp$	(IH on 2)
	5	$Q \vdash_C (\Gamma)^\perp; (\Delta')^\perp, \Lambda', x : A$	(IH on 3)
	6	$(\nu x)(P \mid Q) \vdash_C (\Gamma)^\perp; (\Delta)^\perp, (\Delta')^\perp, \Lambda, \Lambda'$	([CUT] on 4 and 5)
• [CUTRR]	1	$\Gamma; \Delta, \Delta' \vdash (\nu x)(P \mid Q) :: \Lambda, \Lambda'$	(assumption)
	2	$\Gamma; \Delta \vdash P :: \Lambda, x : A$	
	3	$\Gamma; \Delta' \vdash Q :: \Lambda', x : A^\perp$	(inversion on 1)
	4	$P \vdash_C (\Gamma)^\perp; (\Delta)^\perp, \Lambda, x : A$	(IH on 2)
	5	$Q \vdash_C (\Gamma)^\perp; (\Delta')^\perp, \Lambda', x : A^\perp$	(IH on 3)
	6	$(\nu x)(P \mid Q) \vdash_C (\Gamma)^\perp; (\Delta)^\perp, (\Delta')^\perp, \Lambda, \Lambda'$	([CUT] on 4 and 5)
• [CUTLL]	1	$\Gamma; \Delta, \Delta' \vdash (\nu x)(P \mid Q) :: \Lambda, \Lambda'$	(assumption)
	2	$\Gamma; \Delta, x : A \vdash P :: \Lambda$	
	3	$\Gamma; \Delta', x : A^\perp \vdash Q :: \Lambda'$	(inversion on 1)
	4	$P \vdash_C (\Gamma)^\perp; (\Delta)^\perp, \Lambda, x : A^\perp$	(IH on 2)
	5	$Q \vdash_C (\Gamma)^\perp; (\Delta')^\perp, \Lambda', x : A$	(IH on 3)
	6	$(\nu x)(P \mid Q) \vdash_C (\Gamma)^\perp; (\Delta)^\perp, (\Delta')^\perp, \Lambda, \Lambda'$	([CUT] on 4 and 5)

• [CUT!R]	1	$\Gamma;\Delta \vdash (\nu u)(P \mid !u(x).Q) :: \Lambda$	(assumption)
	2	$\Gamma, u : A;\Delta \vdash P :: \Lambda$	
	3	$\Gamma;\emptyset \vdash Q :: x : A$	(inversion on 1)
	4	$P \vdash_{\mathcal{C}} (\Gamma)^{\perp}, u : A^{\perp};(\Delta)^{\perp}, \Lambda$	(IH on 2)
	5	$Q \vdash_{\mathcal{C}} (\Gamma)^{\perp};x : A$	(IH on 3)
	6	$(\nu u)(P \mid !u(x).Q) \vdash_{\mathcal{C}} (\Gamma)^{\perp};(\Delta)^{\perp}, \Lambda$	([CUT?R] on 4 and 5)
• [CUT!L]	1	$\Gamma;\Delta \vdash (\nu u)(!u(x).P \mid Q) :: \Lambda$	(assumption)
	2	$\Gamma;\emptyset \vdash P :: x : A$	
	3	$\Gamma, u : A;\Delta \vdash Q :: \Lambda$	(inversion on 1)
	4	$P \vdash_{\mathcal{C}} (\Gamma)^{\perp};x : A$	(IH on 2)
	5	$Q \vdash_{\mathcal{C}} (\Gamma)^{\perp}, u : A^{\perp};(\Delta)^{\perp}, \Lambda$	(IH on 3)
	6	$(\nu u)(!u(x).P \mid Q) \vdash_{\mathcal{C}} (\Gamma)^{\perp};(\Delta)^{\perp}, \Lambda$	([CUT?L] on 4 and 5)
• [CUT?R]	1	$\Gamma;\Delta \vdash (\nu u)(P \mid !u(x).Q) :: \Lambda$	(assumption)
	2	$\Gamma, u : A;\Delta \vdash P :: \Lambda$	
	3	$\Gamma;x : A^{\perp} \vdash Q :: \emptyset$	(inversion on 1)
	4	$P \vdash_{\mathcal{C}} (\Gamma)^{\perp}, u : A^{\perp};(\Delta)^{\perp}, \Lambda$	(IH on 2)
	5	$Q \vdash_{\mathcal{C}} (\Gamma)^{\perp};x : A$	(IH on 3)
	6	$(\nu u)(P \mid !u(x).Q) \vdash_{\mathcal{C}} (\Gamma)^{\perp};(\Delta)^{\perp}, \Lambda$	([CUT?R] on 4 and 5)
• [CUT?L]	1	$\Gamma;\Delta \vdash (\nu u)(!u(x).P \mid Q) :: \Lambda$	(assumption)
	2	$\Gamma;x : A^{\perp} \vdash P :: \emptyset$	
	3	$\Gamma, u : A;\Delta \vdash Q :: \Lambda$	(inversion on 1)
	4	$P \vdash_{\mathcal{C}} (\Gamma)^{\perp};x : A$	(IH on 2)
	5	$Q \vdash_{\mathcal{C}} (\Gamma)^{\perp}, u : A^{\perp};(\Delta)^{\perp}, \Lambda$	(IH on 3)
	6	$(\nu u)(!u(x).P \mid Q) \vdash_{\mathcal{C}} (\Gamma)^{\perp};(\Delta)^{\perp}, \Lambda$	([CUT?L] on 4 and 5)

($\mathcal{C} \subseteq \mathcal{U}$) Take any $P \in \mathcal{C}$. Then, there are Γ, Δ s.t. $P \vdash_{\mathcal{C}} \Gamma;\Delta$. By showing that this implies $(\Gamma)^{\perp};\emptyset \vdash P :: \Delta$, we have $P \in \mathcal{U}$. As per the second item of Theorem 2.5, Rules $[\curvearrowleft]$ and $[\curvearrowright]$ from Definition 2.6 are admissible in πULL. Therefore, it suffices to show $(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowleft} P :: \Delta$. We do so by induction on the structure of the proof of $P \vdash_{\mathcal{C}} \Gamma;\Delta$.

• [ID]	1	$[x \leftrightarrow y] \vdash_{\mathcal{C}} \Gamma;x : A, y : A^{\perp}$	(assumption)
	2	$(\Gamma)^{\perp};x : A^{\perp} \vdash_{\curvearrowleft} [x \leftrightarrow y] :: y : A^{\perp}$	([IDR])
	3	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowleft} [x \leftrightarrow y] :: x : A, y : A^{\perp}$	($[\curvearrowright]$)
• [$\mathbf{1}$]	1	$x\langle\rangle.\mathbf{0} \vdash_{\mathcal{C}} \Gamma;x : \mathbf{1}$	(assumption)
	2	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowleft} x\langle\rangle.\mathbf{0} :: x : \mathbf{1}$	([$\mathbf{1}$R])
• [$\perp$]	1	$x().P \vdash_{\mathcal{C}} \Gamma;\Delta, x : \perp$	(assumption)
	2	$P \vdash_{\mathcal{C}} \Gamma;\Delta$	(inversion 1)
	3	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowleft} P :: \Delta$	(IH on 2)
	4	$(\Gamma)^{\perp};x : \mathbf{1} \vdash_{\curvearrowleft} x().P :: \Delta$	([$\mathbf{1}$L] on 3)
	5	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowleft} x().P :: \Delta, x : \perp$	($[\curvearrowright]$ on 4)
• [$\otimes$]	1	$(\nu y)x\langle y\rangle.(P \mid Q) \vdash_{\mathcal{C}} \Gamma;\Delta, \Delta', x : A \otimes B$	(assumption)
	2	$P \vdash_{\mathcal{C}} \Gamma;\Delta, y : A$	
	3	$Q \vdash_{\mathcal{C}} \Gamma;\Delta', x : B$	(inversion on 1)

	4	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowright} P :: \Delta, y : A$	(IH on 2)
	5	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowright} Q :: \Delta', x : B$	(IH on 3)
	6	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowright} (\nu y)x\langle y\rangle(P \mid Q) :: \Delta, \Delta', x : A \otimes B$	([⊗R] on 4 and 5)
• [⅋]	1	$x(y).P \vdash_C \Gamma; \Delta, x : A \parr B$	(assumption)
	2	$P \vdash_C \Gamma; \Delta, y : A, x : B$	(inversion on 1)
	3	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowright} P :: \Delta, y : A, x : B$	(IH on 2)
	4	$(\Gamma)^{\perp}; y : A^{\perp} \vdash_{\curvearrowright} P :: \Delta, x : B$	([↶] on 3)
	5	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowright} x(y).P :: \Delta, x : \underbrace{A^{\perp} \multimap B}_{A \parr B}$	([⊸R] on 4)
• [⊕]	1	$x \lhd j.P \vdash_C \Gamma; \Delta, x : \oplus\{i : A_i\}_{i \in I}$	(assumption)
	2	$P \vdash_C \Gamma; \Delta, x : A_j$	
	3	$j \in I$	(inversion on 1)
	4	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowright} P :: \Delta, x : A_j$	(IH on 2)
	5	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowright} x \lhd j.P :: \Delta, x : \oplus\{i : A_i\}_{i \in I}$	([⊕R] on 4 and 3)
• [&]	1	$x \rhd \{i : P_i\}_{i \in I} \vdash_C \Gamma; \Delta, x : \&\{i : A_i\}_{i \in I}$	(assumption)
	2	$\forall i \in I.\ P_i \vdash_C \Gamma; \Delta, x : A_i$	(inversion on 1)
	3	$\forall i \in I.\ (\Gamma)^{\perp};\emptyset \vdash_{\curvearrowright} P_i :: \Delta, x : A_i$	(IH on 2)
	4	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowright} x \rhd \{i : P_i\}_{i \in I} :: \Delta, x : \&\{i : A_i\}_{i \in I}$	([&R] on 3)
• [COPY]	1	$(\nu x)u\langle x\rangle.P \vdash_C \Gamma, u : A; \Delta$	(assumption)
	2	$P \vdash_C \Gamma, u : A; \Delta, x : A$	(inversion on 1)
	3	$(\Gamma)^{\perp}, u : A^{\perp};\emptyset \vdash_{\curvearrowright} P :: \Delta, x : A$	(IH on 2)
	4	$(\Gamma)^{\perp}, u : A^{\perp}; x : A^{\perp} \vdash_{\curvearrowright} P :: \Delta$	([↶] on 3)
	5	$(\Gamma)^{\perp}, u : A^{\perp};\emptyset \vdash_{\curvearrowright} (\nu x)u\langle x\rangle.P :: \Delta$	([COPYL] on 4)
• [!]	1	$!x(y).P \vdash_C \Gamma; x : !A$	(assumption)
	2	$P \vdash_C \Gamma; y : A$	(inversion on 1)
	3	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowright} P :: y : A$	(IH on 2)
	4	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowright} !x(y).P :: x : !A$	([!R] on 3)
• [?]	1	$P\{x/u\} \vdash_C \Gamma; \Delta, x : ?A$	(assumption)
	2	$P \vdash_C \Gamma, u : A; \Delta$	(inversion on 1)
	3	$(\Gamma)^{\perp}, u : A^{\perp};\emptyset \vdash_{\curvearrowright} P :: \Delta$	(IH on 2)
	4	$(\Gamma)^{\perp}; x : !A^{\perp} \vdash_{\curvearrowright} P\{x/u\} :: \Delta$	([!L] on 3)
	5	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowright} P\{x/u\} :: \Delta, x : ?A$	([↷] on 4)
• [CUT]	1	$(\nu x)(P \mid Q) \vdash_C \Gamma; \Delta, \Delta'$	(assumption)
	2	$P \vdash_C \Gamma; \Delta, x : A$	
	3	$Q \vdash_C \Gamma; \Delta', x : A^{\perp}$	(inversion on 1)
	4	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowright} P :: \Delta, x : A$	(IH on 2)
	5	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowright} Q :: \Delta', x : A^{\perp}$	(IH on 3)
	6	$(\Gamma)^{\perp}; x : A \vdash_{\curvearrowright} Q :: \Delta'$	([↶] on 5)
	7	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowright} (\nu x)(P \mid Q) :: \Delta, \Delta'$	([CUTRL] on 4 and 6)

• [CUT?R]	1	$(\nu u)(P \mid !u(x).Q) \vdash_C \Gamma;\Delta$	(assumption)
	2	$P \vdash_C \Gamma, u : A;\Delta$	
	3	$Q \vdash_C \Gamma; x : A^{\perp}$	(inversion on 1)
	4	$(\Gamma)^{\perp}, u : A^{\perp};\emptyset \vdash_{\curvearrowright} P :: \Delta$	(IH on 2)
	5	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowright} Q :: x : A^{\perp}$	(IH on 3)
	6	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowright} (\nu u)(P \mid !u(x).Q) :: \Delta$	([CUT!R])
• [CUT?L]	1	$(\nu u)(!u(x).P \mid Q) \vdash_C \Gamma;\Delta$	(assumption)
	2	$P \vdash_C \Gamma; x : A^{\perp}$	
	3	$Q \vdash_C \Gamma, u : A;\Delta$	(inversion on 1)
	4	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowright} P :: x : A^{\perp}$	(IH on 2)
	5	$(\Gamma)^{\perp}, u : A^{\perp};\emptyset \vdash_{\curvearrowright} Q :: \Delta$	(IH on 3)
	6	$(\Gamma)^{\perp};\emptyset \vdash_{\curvearrowright} (\nu u)(!u(x).P \mid Q) :: \Delta$	([CUT!L])

This concludes the proof of Theorem 3.1. □

We now turn our attention to $\mathcal{I}$, the class of processes typable under the intuitionistic interpretation. The observation by Caires, Pfenning and Toninho in [12] entails that $\mathcal{I}$ (πILL-typable processes) should be a strict subset of $\mathcal{C}$ (πCLL*-typable processes). We formalize this fact by characterizing the intuitionistic fragment of πULL (which coincides with πILL) by limiting its typing rules and by showing that the class of processes typable in this fragment coincides with $\mathcal{I}$ (Theorem 3.2). It then follows that this class of processes is strictly contained in $\mathcal{U}^{\curvearrowright}$ (πULL-typable processes; Theorem 3.3). Since we have just shown that $\mathcal{U}^{\curvearrowright} = \mathcal{C}$, this formalizes the fact that $\mathcal{I}$ is a strict subset of $\mathcal{C}$.

Theorem 3.2 below formalizes two equivalent characterizations of the intuitionistic fragment of πULL. One characterization is based on the limited form of duality of πILL: the Rules [$\curvearrowleft$] and [$\curvearrowright$] in Definition 2.6 may not be used. The other characterization is based on the restricted two-sided form of πILL sequents: the rules in Fig. 2 are limited to have exactly one channel/type pair on the right. This requires the following auxiliary definition:

Definition 3.2 *(r-degree).* The *r-degree* of a πULL sequent $\Gamma;\Delta \vdash P :: \Lambda$ is the size of Λ. We say this sequent has *r*-degree $|\Lambda|$.

We now have:

Theorem 3.2. *Given a process P, the following are equivalent:*

1. *there are Γ, Δ, x and A such that $\Gamma;\Delta \vdash_{\mathcal{I}} P :: x : A$;*
2. *there are Γ, Δ and Λ such that $\Gamma;\Delta \vdash P :: \Lambda$ where all sequents in its proof have r-degree 1;*
3. *there are Γ, Δ and Λ such that $\Gamma;\Delta \vdash_{\curvearrowright} P :: \Lambda$ where its proof never uses Rules [$\curvearrowleft$] and [$\curvearrowright$].*

Proof. We first argue that items 1 and 2 are equivalent; then, we argue that items 2 and 3 imply each other:

(Equivalence of items 1 and 2) We show that restricting the sequents in πULL to an *r*-degree of 1 entails the limitation of its rules to a strict subset: those marked with $*$ in Fig. 2. This set of rules coincides with the set of rules for πILL in Fig. 4. Hence, a proof of typability in πILL (item 1) can be replicated in πULL with *r*-degree 1 (item 2) and vice versa. Because the proof for each rule follows the same pattern, we only detail two cases: one without $*$ and one with $*$.

- Rule [⅋L] is not $*$-marked. Suppose we have a proof of typability in πULL with *r*-degree 1 and suppose this includes an application of [⅋L]. By assumption, this application's antecedents have *r*-degree 1, so its consequence must have *r*-degree 2. This contradicts the assumption that all sequents have *r*-degree 1, showing that Rule [⅋L] is not usable in this fragment of πULL.
- Rule [$\multimap$L] is $*$-marked. By assumption, in any application of this rule its antecedents would have *r*-degree 1. Then, its consequence also has *r*-degree 1, so this rule can be used without problems.

(Item 2 implies item 3) By Definition 2.6, the set of rules of $\pi\mathsf{ULL}_{\curvearrowright}$ consists of only [$\curvearrowleft$], [$\curvearrowright$], and the $*$-marked rules in Fig. 2. In $\pi\mathsf{ULL}_{\curvearrowright}$, the only rules that alter the *r*-degrees in judgments are [$\curvearrowleft$] and [$\curvearrowright$]. Hence, if these rules may not be used, proofs of typability in $\pi\mathsf{ULL}_{\curvearrowright}$ have a constant *r*-degree. Since all axioms of $\pi\mathsf{ULL}_{\curvearrowright}$ (the $*$-marked Axioms [IDR] and [**1**R] in Fig. 2) have *r*-degree 1, this constant *r*-degree is 1. Therefore, a proof of typability in $\pi\mathsf{ULL}_{\curvearrowright}$ without using [$\curvearrowleft$] and [$\curvearrowright$] (item 3) can be replicated exactly in πULL with *r*-degree restricted to 1 (item 2).

(Item 3 implies item 2) The proof of equivalence of items 1 and 2 above shows that with *r*-degree 1 the only usable rules in πULL are those marked with $*$ in Fig. 2. By Definition 2.6, without Rules [$\curvearrowleft$] and [$\curvearrowright$], $\pi\mathsf{ULL}_{\curvearrowright}$ consists only of these $*$-marked rules.

Hence, a proof of typability in πULL with r-degree restricted to 1 (item 2) can be replicated exactly in $\pi\mathsf{ULL}_{\curvearrowright}$ without using [$\curvearrowleft$] and [$\curvearrowright$] (item 3). □

We may now state our final result:

Theorem 3.3. $\mathcal{I} \subsetneq \mathcal{U}^{\curvearrowright}$.

Proof. Theorem 3.2 ensures that $\mathcal{I} \subseteq \mathcal{U}^{\curvearrowright}$. To show that this inclusion is strict, we give a process $P \in \mathcal{U}^{\curvearrowright}$ such that $P \notin \mathcal{I}$. Assuming a proof for $u : B; \emptyset \vdash P' :: z : A$, let $P = x(y).!y(z).P'\{x/u\}$; there are precisely three ways to prove $P \in \mathcal{U}^{\curvearrowright}$: they follow from the three ways to infer a receive in πULL ([⅋R], [⊸R], and [⊗L]). The proofs are as follows:

$$\dfrac{\dfrac{\dfrac{u : B; \emptyset \vdash P' :: z : A}{u : B; \emptyset \vdash !y(z).P' :: y : !A}\,[!\mathrm{R}]}{\emptyset; \emptyset \vdash !y(z).P'\{x/u\} :: y : !A, x : ?B^{\perp}}\,[?\mathrm{R}]}{\emptyset; \emptyset \vdash x(y).!y(z).P'\{x/u\} :: x : (!A) \parr (?B^{\perp})}\,[\parr\mathrm{R}]$$

$$\dfrac{\dfrac{\dfrac{\dfrac{u : B; \emptyset \vdash P' :: z : A}{u : B; z : A^{\perp} \vdash P' :: \emptyset}\,[\curvearrowleft]}{u : B; y : ?A^{\perp} \vdash !y(z).P' :: \emptyset}\,[?\mathrm{L}]}{\emptyset; y : ?A^{\perp} \vdash !y(z).P'\{x/u\} :: x : ?B^{\perp}}\,[?\mathrm{R}]}{\emptyset; \emptyset \vdash x(y).!y(z).P'\{x/u\} :: x : (?A^{\perp}) \multimap (?B^{\perp})}\,[\multimap\mathrm{R}]$$

$$\dfrac{\dfrac{\dfrac{\dfrac{u : B; \emptyset \vdash P' :: z : A}{u : B; z : A^{\perp} \vdash P' :: \emptyset}\,[\curvearrowleft]}{u : B; y : ?A^{\perp} \vdash !y(z).P' :: \emptyset}\,[?\mathrm{L}]}{\emptyset; y : ?A^{\perp}, x : !B \vdash !y(z).P'\{x/u\} :: \emptyset}\,[!\mathrm{L}]}{\emptyset; x : (?A^{\perp}) \otimes (!B) \vdash x(y).!y(z).P'\{x/u\} :: \emptyset}\,[\otimes\mathrm{L}]$$

Clearly, all these proofs contain judgments of r-degree different from 1 (first case) or require using [$\curvearrowleft$]/[$\curvearrowright$] (second and third cases). Hence, by Theorem 3.2, $P \notin \mathcal{I}$. □

4. Analysis

Now that we have established characterizations for the intuitionistic and classical fragments of πULL, we analyze the meaning of these results and discuss possible extensions.

Our analysis is twofold. First, we consider the informal observation by Caires, Pfenning, and Toninho [12] that intuitionistic type systems enforce the locality principle (§ 4.1); unlike the classical formulation, the intuitionistic fragment of πULL (and thus πILL) has a partial form of duality, which ensures that typability guarantees locality. Next, we discuss how πULL can support alternative, more expressive forms of parallel composition and restriction, and how such extensions transfer to classical or intuitionistic type systems (§ 4.2); we will see that the complete duality of classical type systems allows for such extensions, while the rely-guarantee reading of intuitionistic type systems cannot account for them.

4.1. Locality

Locality is a well-known principle in concurrency research [40]. The idea is that freshly created channels are *local*. Local channels are *mutable*, in the sense that they can be used for receives. Once a channel has been transmitted to another location, it becomes *non-local*, and thus *immutable*: it can only be used for sends—receives are no longer allowed. This makes locality particularly relevant for giving formal semantics to distributed programming languages; a prime example is the *join calculus* [21], whose theory relies on (and is deeply influenced by) the locality principle [20].

Locality also makes an appearance in other contexts. Honda and Laurent's [27] correspondence between a typed π-calculus and polarized proof-nets enforces locality due to receives having negative polarity while received names may only be of positive polarity. Also, Dal Lago et al.'s [17] typed π-calculus with intersection types goes further: processes are *hyperlocalized*, meaning that there may be no receives on free channels after a prior receive at all.

Neither the intuitionistic nor the classical interpretation guarantee full-fledged locality through typing: both systems allow receives on previously received channels. The exception is *replicated receive*, which is used to define a shared channel that can continuously receive linear channels over which to perform a service. The intuitionistic interpretation guarantees locality for shared channels; in other words, πILL cannot type replicated receives on non-local channels.

The following example, taken from the work by Caires, Pfenning and Toninho [12], is typable in $\pi\mathsf{CLL}^{\star}$ but not in πILL:

$$(\nu x)(x(y).!y(z).P \mid (\nu q)x\langle q\rangle.Q)$$

Consider the left process in the parallel composition, $x(y).!y(z).P$. It first receives a channel y over channel x; then, it uses y for replicated receive, thus defining it as a shared channel. In $\pi\mathsf{CLL}^{\star}$, channel x has type $!A \parr B$. The intuitionistic variant of this type is $(!A)^{\perp} \multimap B = ?A^{\perp} \multimap B$ on the right of the typing judgment, and $?A^{\perp} \otimes B^{\perp}$ on the left. It is impossible to type a process with a channel of such a type in πILL, because '?' is not a connective in the intuitionistic system.

The fact that πILL cannot type non-local shared channels—due to the absence of a dual for '!'—suggests that there should be another kind of non-local channels that πILL cannot type: there is no dual for $\mathbf{1}$. Indeed, πILL cannot type empty sends on previously received channels, a feature that is not observed in [12]. The following example is typable in $\pi\mathsf{CLL}^{\star}$ but not in πILL:

$$x(y).x().y\langle\rangle.\mathbf{0}$$

In $\pi\mathsf{CLL}^{\star}$, the type of x in this process is $\mathbf{1} \parr \bot$. The intuitionistic variant of this type is $\mathbf{1}^{\bot} \multimap \bot = \bot \multimap \bot$ on the right of the typing judgment, and $\bot \otimes \mathbf{1}$ on the left. This process is not typable in πILL, because $\bot$ is not an intuitionistic proposition and so the type system has no rules to type $\bot$ channels. We make this observation more precise by giving an alternative proof to Theorem 3.3 ($\mathcal{I} \subsetneq \mathcal{U}$) based on this observation:

Alternative proof of Theorem 3.3. By Theorem 3.2, it is sufficient to give $P \in \mathcal{U}$ such that $P \notin \mathcal{I}$. Let $P := x(y).x().y\langle\rangle.\mathbf{0}$. There are three ways to prove that $P \in \mathcal{U}$:

$$\dfrac{\dfrac{\dfrac{}{\Gamma; y : \bot \vdash y\langle\rangle.\mathbf{0} :: \emptyset}\,[\bot\mathrm{L}]}{\Gamma; y : \bot, x : \mathbf{1} \vdash x().y\langle\rangle.\mathbf{0} :: \emptyset}\,[\mathbf{1}\mathrm{L}]}{\Gamma; x : \bot \otimes \mathbf{1} \vdash x(y).x().y\langle\rangle.\mathbf{0} :: \emptyset}\,[\otimes\mathrm{L}] \qquad \dfrac{\dfrac{}{\Gamma; \emptyset \vdash y\langle\rangle.\mathbf{0} :: y : \mathbf{1}}\,[\mathbf{1}\mathrm{R}]}{\Gamma; \emptyset \vdash x().y\langle\rangle.\mathbf{0} :: y : \mathbf{1}, x : \bot}\,[\bot\mathrm{R}]}{\Gamma; \emptyset \vdash x(y).x().y\langle\rangle.\mathbf{0} :: x : \mathbf{1} \parr x : \bot}\,[\parr\mathrm{R}] \qquad \dfrac{\dfrac{\dfrac{}{\Gamma; y : \bot \vdash y\langle\rangle.\mathbf{0} :: \emptyset}\,[\bot\mathrm{L}]}{\Gamma; y : \bot \vdash x().y\langle\rangle.\mathbf{0} :: x : \bot}\,[\bot\mathrm{R}]}{\Gamma; \emptyset \vdash x(y).x().y\langle\rangle.\mathbf{0} :: x : \bot \multimap \bot}\,[\multimap\mathrm{RL}]$$

Clearly, all these proofs contain judgments of r-degree different from 1. Hence, by Theorem 3.2, $P \notin \mathcal{I}$. □

Based on these two proofs for Theorem 3.3, we have corroborated Caires et al.'s observation on locality, and extended it to the case of empty sends and receives.

Judgments, revisited As already discussed, '$\bot$' and '?' are not propositions in πILL; the absence of rules for them in πILL is an inherent consequence of the form of its judgments. As shown by Theorem 3.2, rules for '?' and '$\bot$' are an impossibility when judgments are required to have exactly one assignment (channel/proposition) on the right. Theorem 3.2 also shows this is closely related to duality, which is not total in the intuitionistic interpretation (i.e., the duals for '$\mathbf{1}$' and '!' given by Definition 2.2 are not propositions in πILL). In the classical case, the type system is *symmetrical* (as shown by a support for [↶]/[↷]-rules), while the intuitionistic type system is *asymmetrical*.

Concerning '$\bot$', the judgmental presentation of intuitionistic linear logic in [6] defines an alternative approach, which allows deriving '$\bot$' as 'multiplicative contradiction' by using a judgmental account of possibility under which resources are used zero times.

4.2. Parallel composition and restriction

The type systems we have discussed so far are rather restrictive in terms of how processes can be composed and connected: there is only the cut-rule which composes and connects two processes that have exactly one channel of dual type in common. Indeed, the cut-rule jointly handles constructs for parallel composition and restriction, which contrasts with many non-logical type systems for the π-calculus (e.g., [36,35,54]) where parallel composition and restriction typically have a dedicated rule each.

In this section we discuss extensions to the type systems that decompose cut into two separate rules: mix for parallel composition, and cycle for channel connection (restriction). As we will see, these notions form another clear distinction point between Curry-Howard interpretations of classical and intuitionistic linear logic.

4.2.1. Independent parallel composition

Caires et al. [12] discuss an alternative form of parallel composition. In the intuitionistic interpretation, the so-called *independent* parallel composition connects (i) an arbitrary process Q with (ii) a process P with a channel of type $\mathbf{1}$ on the right; it is derivable in the *silent* interpretation of $\mathbf{1}$, in which Axioms [$\mathbf{1}$R] and [$\bot$L] type the inactive process $\mathbf{0}$ and Rules [$\bot$R] and [$\mathbf{1}$L] leave processes untouched. This way, the rules for $\mathbf{1}$ would be the following (the rules for $\bot$ are similar):

$$\dfrac{}{\Gamma; \emptyset \vdash \mathbf{0} :: x : \mathbf{1}}\,[\mathbf{1}\mathrm{R}] \qquad \dfrac{\Gamma; \Delta \vdash P :: \Lambda}{\Gamma; \Delta, x : \mathbf{1} \vdash P :: \Lambda}\,[\mathbf{1}\mathrm{L}]$$

In this case, the correspondence relies on structural congruences in processes and proofs (suitably extended), rather than on reduction. To obtain independent parallel composition, one uses Rule [CUT] to connect the right channel of P with a corresponding (dual) channel in Q, which is exposed on the left using Rule [$\mathbf{1}$L]:

$$\dfrac{\Gamma; \Delta \vdash_{\mathcal{I}} P :: z : \mathbf{1} \qquad \dfrac{\Gamma; \Delta' \vdash_{\mathcal{I}} Q :: x : C}{\Gamma; \Delta', z : \mathbf{1} \vdash_{\mathcal{I}} Q :: x : C}\,[\mathbf{1}\mathrm{L}]}{\Gamma; \Delta, \Delta' \vdash_{\mathcal{I}} (\nu z)(P \mid Q) :: x : C}\,[\mathrm{CUTRL}]$$

The requirement that process P above has a channel of type $\mathbf{1}$ on the right is rather restrictive: only left-rules can be used for typing, so P must be a process that relies on multiple behaviors but can offer a behavior of type $\mathbf{1}$. Related to this constraint, Caires et al. show

that in the silent interpretation $\Gamma; \Delta \vdash_I P :: x : A$ implies $\Gamma; \Delta, x : A^\perp \vdash_I P :: z : \mathbf{1}$, for some fresh z, provided that A is exponential-free and $\mathbf{1}^\perp = \mathbf{1}$ (cf. also below) [12, Prop. 5.1]. However, this "movement" from the right to the left is only possible under the absence of the identity axiom, which is in turn necessary when considering, e.g., behavioral polymorphism [9]. Indeed, given any type A we can prove $\emptyset; x : A \vdash_I [x \leftrightarrow y] :: y : A$ using the identity axiom, but there is no way to prove $\emptyset; x : A, y : A^\perp \vdash_I [x \leftrightarrow y] :: z : \mathbf{1}$.

4.2.2. Parallel composition: mix

Girard [23] discusses an extension of linear logic that allows the combination of two independent proofs. Wadler gives a Curry-Howard interpretation of this rule as the parallel composition of two processes that have no channels in common [56]. In [5], Caires complements the extension with an axiom that types the inactive process $\mathbf{0}$ with an empty context.

We now define $\pi\mathsf{CLL}^\star_+$, which is $\pi\mathsf{CLL}^\star$ extended with this form of parallel composition. The only change is the addition of the following rules:

$$\frac{P \vdash_C \Gamma; \Delta \qquad Q \vdash_C \Gamma; \Delta'}{P \mid Q \vdash_C \Gamma; \Delta, \Delta'}\ [\textsc{mix}] \qquad\qquad \frac{}{\mathbf{0} \vdash_C \Gamma; \emptyset}\ [\textsc{empty}]$$

We straightforwardly define $\pi\mathsf{ULL}_+$ as a similar extension of $\pi\mathsf{ULL}$ by adding the following rules:

$$\frac{\Gamma; \Delta \vdash P :: \Lambda \qquad \Gamma; \Delta' \vdash Q :: \Lambda'}{\Gamma; \Delta, \Delta' \vdash P \mid Q :: \Lambda, \Lambda'}\ [\textsc{mix}] \qquad\qquad \frac{}{\Gamma; \emptyset \vdash \mathbf{0} :: \emptyset}\ [\textsc{empty}]$$

It is easy to see that Theorem 3.1—the equivalence of $\pi\mathsf{CLL}^\star$ and $\pi\mathsf{ULL}$—still holds for $\pi\mathsf{CLL}^\star_+$ and $\pi\mathsf{ULL}_+$. Following the reasoning of Theorem 3.2—the characterization of $\pi\mathsf{ILL}$ in terms of $\pi\mathsf{ULL}$—, Rules [MIX] and [EMPTY] do not belong to the intuitionistic fragment of $\pi\mathsf{ULL}_+$: Rule [EMPTY] has r-degree 0, and if the antecedents of Rule [MIX] both have r-degree 1 then its consequence would have r-degree 2. This leads us to conclude that the more flexible ways of composition induced by [MIX] cannot be supported by intuitionistic systems.

The equivalence of $\mathbf{1}$ *and* $\perp$ Caires [5] notes that, in the classical setting, Rules [MIX] and [EMPTY] make it possible to prove $\perp \multimap \mathbf{1}$ and $\mathbf{1} \multimap \perp$ (where $\multimap$ denotes linear implication). Hence, it is possible to consider $\mathbf{1}$ and $\perp$ equivalent, writing a single symbol (say, '•') for either—very similar to the singular, self-dual type end in standard session types. The work of Atkey et al. [1] is also relevant, as it develops a detailed treatment of the conflation of $\perp$ and $\mathbf{1}$. Conflation is not always desirable: the work of Qian et al. [47] addresses the shortcomings of considering [MIX] for faithfully representing stateful client-server behaviors.

4.2.3. Channel connection: cycle

The cut-rule only allows us to connect two processes on a single channel. Although this neatly guarantees deadlock freedom, it prevents many deadlock-free processes from being typable (see [18] for comparisons between these classes of processes). For example, we can construct a process $(\nu x)(\nu y)(P \mid Q)$ where

$$P := (\nu u)x\langle u\rangle.(y(v).(u().v().\mathbf{0} \mid y\langle\rangle.\mathbf{0}) \mid x\langle\rangle.\mathbf{0}) \text{ and}$$

$$Q := x(w).(\nu z)y\langle z\rangle.(x().z\langle\rangle.\mathbf{0} \mid y().w\langle\rangle.\mathbf{0})$$

This process is deadlock free, but not typable using cut.

Connecting on multiple channels at once does have the danger of introducing the *circular dependencies* between sessions that are at the heart of deadlocked processes. For example, suppose that we replace P with P', where we swap the send on x and the receive on y:

$$P' := y(v).(\nu u)x\langle u\rangle.(u().v().\mathbf{0} \mid y\langle\rangle.\mathbf{0} \mid x\langle\rangle.\mathbf{0}).$$

Now, the composed and connected process would be stuck with P' waiting for a receive on y and Q for a receive on x.

Dardha and Gay [14] present a session type system based on classical linear logic in which they replace the cut-rule with a rule called [CYCLE], which connects two channels of dual types in the same judgment. This new rule has a side-condition that allows their type system to maintain deadlock freedom. We can define $\pi\mathsf{CLL}^\star_{+\!+}$ as an extension of $\pi\mathsf{CLL}^\star_+$ with the cycle-rule. Since we are specifically interested in this form of channel restriction, below we abstract away from Dardha and Gay's side-condition by simply writing ϕ.

Note that we first require some modifications to restriction and reduction, based on [54]. Restriction in $\pi\mathsf{CLL}^\star_{+\!+}$ should involve two channel names, i.e. writing $(\nu xy)P$ instead of $(\nu x)P$, because the involved channels appear in the same judgment and thus need to be uniquely named. Indeed, $(\nu xy)P$ says that x and y are the two endpoints of the same channel in P.

$$\frac{P \vdash_C \Gamma; \Delta, x : A, y : A^\perp \qquad \phi}{(\nu xy)P \vdash_C \Gamma; \Delta}\ [\textsc{cycle}]$$

Defining $\pi\mathsf{ULL}_{\#}$ as a similar extension of $\pi\mathsf{ULL}_{+}$ is again straightforward. Similarly to the cut-rules in Fig. 3, we add four rules that operate on different sides of the typing judgment:

$$\text{[CYCLERL]}\ \frac{\Gamma;\Delta, x:A \vdash P :: \Lambda, y:A \qquad \phi}{\Gamma;\Delta \vdash (\nu yx)P :: \Lambda} \qquad \text{[CYCLELR]}\ \frac{\Gamma;\Delta, x:A \vdash P :: \Lambda, y:A \qquad \phi}{\Gamma;\Delta \vdash (\nu xy)P :: \Lambda} \qquad \text{[CYCLERR]}\ \frac{\Gamma;\Delta \vdash P :: \Lambda, x:A, y:A^{\perp} \qquad \phi}{\Gamma;\Delta \vdash (\nu xy)P :: \Lambda}$$

$$\text{[CYCLELL]}\ \frac{\Gamma;\Delta, x:A, y:A^{\perp} \vdash P :: \Lambda \qquad \phi}{\Gamma;\Delta \vdash (\nu xy)P :: \Lambda}$$

Again, it is easy to see that Theorem 3.1 still holds for $\pi\mathsf{CLL}^{\star}_{\#}$ and $\pi\mathsf{ULL}_{\#}$. Interestingly, by virtue of Theorem 3.2, Rule [CYCLELL] is the only rule allowed in the intuitionistic fragment of $\pi\mathsf{ULL}_{\#}$, because it does not affect the right-hand side of the judgment (as Λ can have r-degree 1). Unfortunately, this rule is useless for $\pi\mathsf{ILL}$: it requires two channels of dual types, but duality is not total in $\pi\mathsf{ILL}$.

Adding mix- and cycle-rules enables a more flexible formulation of structural congruence, as well as more flexible typing of sends. For one, e.g., Rule [$\otimes$R] would only require a single continuation that provides both the payload and the continuation of the send. We can then add structural congruence rules that are found in traditional, untyped settings, such as $P \mid Q \equiv Q \mid P$ and $P \mid \mathbf{0} \equiv P$.

5. Conclusion

Curry-Howard correspondences between linear logic and session types explain how linear logic can provide an ample, principled framework to conceive different typing disciplines for message-passing processes specified in the π-calculus. There is no single canonical interpretation, as there are multiple interpretations, depending on design choices involving, e.g., the logical connectives considered and their respective process operators.

In this context, this paper has pursued a very concrete goal: to formally compare the interpretations of classical and intuitionistic linear logic as session types. The comparison results reported in Section 3 are an indispensable step to consolidating the logical foundations of message-passing concurrency; they also have a number of relevant ramifications, as reported in Section 4.

Our technical approach to this goal relies on a fragment of Girard's Logic of Unity (LU) [24] as a basis to develop $\pi\mathsf{ULL}$: a new session type system that can type all processes typable in both $\pi\mathsf{CLL}^{\star}$ and $\pi\mathsf{ILL}$, which correspond to classical and intuitionistic interpretations, respectively. The linear logic ULL, on which the type system $\pi\mathsf{ULL}$ stands, is an admittedly modest fragment of Girard's LU. Still, we emphasize that $\pi\mathsf{ULL}$ is sufficient for our purposes, as it gives us a fair and rigorous basis for addressing our declared goal of comparing type systems based on different linear logics. The development of computational interpretations for LU, including but going beyond ULL and $\pi\mathsf{ULL}$, is surely an interesting direction but one that lies outside the scope of our work.

In $\pi\mathsf{ILL}$, judgments have a particular reading in which a process *relies* on several channels (on the left of judgments) and *guarantees* a behavior along a single designated channel (on the right). $\pi\mathsf{ULL}$ uses two-sided judgments as well, similar to LU. However, $\pi\mathsf{ULL}$ does not retain $\pi\mathsf{ILL}$'s rely-guarantee reading, because it does not distinguish between the sides of its sequents. The consequence is that $\pi\mathsf{ULL}$ supports a full duality relation, as opposed to $\pi\mathsf{ILL}$. This allows $\pi\mathsf{ULL}$ to mimic the explicit duality in the single-sided $\pi\mathsf{CLL}^{\star}$. On the other hand, restricting the right side of $\pi\mathsf{ULL}$'s judgments to exactly one channel—thus limiting support for duality—characterizes a fragment of $\pi\mathsf{ULL}$'s typing rules that precisely coincides with $\pi\mathsf{ILL}$.

Our results confirm the informal observation by Caires et al. [12] that the difference between session type systems based on classical and intuitionistic linear logic is in the enforcement of locality of shared names. We have not only confirmed that $\pi\mathsf{ILL}$ cannot type processes that do not respect locality for shared channels: a new insight obtained in this paper is that $\pi\mathsf{ILL}$ also forbids empty sends on received channels. Our results show that these constraints are a consequence of the strict requirement that $\pi\mathsf{ILL}$'s typing judgments must have exactly one channel/type on the right. $\pi\mathsf{ULL}$ and $\pi\mathsf{CLL}^{\star}$ do not have such constraints and thus fully support duality.

Our work can be seen as providing a technical answer to long-suspected but fuzzily understood differences between intuitionistic and classical interpretations of linear logic (here represented by $\pi\mathsf{ILL}$ and $\pi\mathsf{CLL}^{\star}$, respectively) concerning the role of duality (implicit in $\pi\mathsf{ILL}$, explicit in $\pi\mathsf{CLL}^{\star}$) and locality of shared channels (enforced by $\pi\mathsf{ILL}$ but not by $\pi\mathsf{CLL}^{\star}$). We believe it is important to give a formal footing to this kind of informal observations. Formally stating the required comparisons is not obvious, and insightful in itself, as implicit assumptions may emerge in the process. In this respect, we adopted $\pi\mathsf{ULL}$ as it provides an effective yardstick for comparisons, independent from both $\pi\mathsf{ILL}$ and $\pi\mathsf{CLL}^{\star}$, and based on an already existing framework (Girard's LU). In future work, it would be interesting to explore other approaches towards our technical results (Theorems 3.1 and 3.3).

If one adopts the stance that a *permissive* type system is also an *expressive* one, then the results from our comparisons (notably, the strict inclusion of $\pi\mathsf{ILL}$ in $\pi\mathsf{ULL}$) indicate that classical linear logic induces a more expressive class of typed processes than intuitionistic linear logic. There is an alternative stance, which considers *less permissive* type systems as being *more precise* than other, more permissive type system. The locality property for shared channels, as enforced by intuitionistic interpretations, is a case in point here. In our view, the connection between permissiveness, expressiveness, and precision is an interesting question, which largely depends on the value of the intended properties. Indeed, the precision of type systems derived from intuitionistic interpretations may not be meaningful in settings/applications where locality is simply not a relevant property. On the other hand, the conditions that make intuitionistic interpretations less permissive can always be imposed on more permissive interpretations as side-conditions,

making them more precise. Finally, as observed by Caires et al. [12], it is surely remarkable that intuitionistic interpretations of session types precisely capture a principle such as locality of shared names, which was known and exploited in different contexts [40] long before these Curry-Howard interpretations were first spelled out [7].

To conclude, we believe that LU and πULL have other useful applications besides the comparison of type systems based on vanilla linear logic; our discussion of more expressive parallel composition and channel connection in Section 4 corroborates this. We have been able to rely on πULL to study the effects of mix- and cycle-rules in the intuitionistic interpretation by transferring extensions of the classical interpretation to πULL and following the methodology of our comparison results. In this case, extensions of intuitionistic interpretations with such rules do not make sense due to the constraints of typing judgments. This makes sense from a logical perspective: intuitionistic argumentation is based on a constructivist philosophy in which putting unrelated things together ([MIX]) and then relating them later ([CYCLE]) may not be acceptable.

CRediT authorship contribution statement

Bas van den Heuvel: Writing – original draft, Formal analysis, Conceptualization. **Jorge A. Pérez:** Writing – original draft, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

We are grateful to Juan C. Jaramillo, Joseph Paulus, and Revantha Ramanayake for helpful discussions. We would also like to thank the anonymous reviewers of PLACES'20 and JLAMP for their detailed suggestions, which were helpful to improve the presentation.

References

[1] Robert Atkey, Sam Lindley, J. Garrett Morris, Conflation confers concurrency, in: Sam Lindley, Conor McBride, Phil Trinder, Don Sannella (Eds.), A List of Successes That Can Change the World: Essays Dedicated to Philip Wadler on the Occasion of His 60th Birthday, in: Lecture Notes in Computer Science, Springer International Publishing, Cham, 2016, pp. 32–55.
[2] Michele Boreale, On the expressiveness of internal mobility in name-passing calculi, Theor. Comput. Sci. 195 (2) (March 1998) 205–226.
[3] Andrew Barber, Gordon D. Plotkin, Dual Intuitionistic Linear Logic, Technical Report ECS-LFCS-96-347, University of Edinburgh, School of Informatics, Laboratory for Foundations of Computer Science, Edinburgh, September 1996.
[4] Stephanie Balzer, Frank Pfenning, Manifest sharing with session types, Proc. ACM Program. Lang. 37 (August 2017) 37:1–37:29.
[5] Luís Caires, Types and logic, concurrency and non-determinism, Technical Report MSR-TR-2014-104, in: Essays for the Luca Cardelli Fest, Microsoft Research, September 2014.
[6] Bor-Yuh Evan Chang, Kaustuv Chaudhuri, Frank Pfenning, A judgmental analysis of linear logic, Technical Report CMU-CS-03-131R, Department of Computer Science, Carnegie Mellon University, November 2003.
[7] Luís Caires, Frank Pfenning, Session types as intuitionistic linear propositions, in: Paul Gastin, François Laroussinie (Eds.), CONCUR 2010 - Concurrency Theory, in: Lecture Notes in Computer Science, Springer, Berlin, Heidelberg, 2010, pp. 222–236.
[8] Luís Caires, Jorge A. Pérez, Linearity, control effects, and behavioral types, in: Hongseok Yang (Ed.), Programming Languages and Systems, in: Lecture Notes in Computer Science, Springer, Berlin, Heidelberg, 2017, pp. 229–259.
[9] Luís Caires, Jorge A. Pérez, Frank Pfenning, Bernardo Toninho, Behavioral polymorphism and parametricity in session-based communication, in: Matthias Felleisen, Philippa Gardner (Eds.), Programming Languages and Systems, Springer, Berlin, Heidelberg, 2013, pp. 330–349.
[10] Luís Caires, Jorge A. Pérez, Frank Pfenning, Bernardo Toninho, Domain-aware session types, in: Fokkink Wan, Rob van Glabbeek (Eds.), 30th International Conference on Concurrency Theory (CONCUR 2019), Dagstuhl, Germany, in: Leibniz International Proceedings in Informatics (LIPIcs), vol. 140, Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2019, pp. 39:1–39:17.
[11] Luís Caires, Frank Pfenning, Bernardo Toninho, Towards concurrent type theory, in: Proceedings of the 8th ACM SIGPLAN Workshop on Types in Language Design and Implementation, ACM, January 2012, pp. 1–12.
[12] Luís Caires, Frank Pfenning, Bernardo Toninho, Linear logic propositions as session types, Math. Struct. Comput. Sci. 26 (3) (March 2016) 367–423.
[13] Henry DeYoung, Luís Caires, Frank Pfenning, Bernardo Toninho, Cut reduction in linear logic as asynchronous session-typed communication, in: Patrick Cégielski, Arnaud Durand (Eds.), Computer Science Logic (CSL'12) - 26th International Workshop/21st Annual Conference of the EACSL, Dagstuhl, Germany, in: Leibniz International Proceedings in Informatics (LIPIcs), vol. 16, Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2012, pp. 228–242.
[14] Ornela Dardha, Simon J. Gay, A new linear logic for deadlock-free session-typed processes, in: Christel Baier, Ugo Dal Lago (Eds.), Foundations of Software Science and Computation Structures, in: Lecture Notes in Computer Science, Springer International Publishing, 2018, pp. 91–109.
[15] Ornela Dardha, Elena Giachino, Davide Sangiorgi, Session types revisited, Inf. Comput. 256 (October 2017) 253–286.
[16] Romain Demangeon, Kohei Honda, Full abstraction in a subtyped pi-calculus with linear types, in: Joost-Pieter Katoen, Barbara König (Eds.), CONCUR 2011 – Concurrency Theory, in: Lecture Notes in Computer Science, Springer, Berlin, Heidelberg, 2011, pp. 280–296.
[17] Ugo Dal Lago, Marc de Visme, Damiano Mazza, Akira Yoshimizu, Intersection types and runtime errors in the pi-calculus, Proc. ACM Program. Lang. 3 (POPL) (January 2019) 7:1–7:29.
[18] Ornela Dardha, Jorge A. Pérez, Comparing type systems for deadlock freedom, J. Log. Algebraic Methods Program. 124 (January 2022) 100717.
[19] Dan Frumin, Emanuele D'Osualdo, Bas van den Heuvel, Jorge A. Pérez, A bunch of sessions: a propositions-as-sessions interpretation of bunched implications in channel-based concurrency, Proc. ACM Program. Lang. 6 (OOPSLA2) (October 2022) 155:841–155:869.
[20] Cédric Fournet, Georges Gonthier, Jean-Jacques Levy, Luc Maranget, Didier Rémy, A calculus of mobile agents, in: Ugo Montanari, Vladimiro Sassone (Eds.), CONCUR '96: Concurrency Theory, in: Lecture Notes in Computer Science, Springer, Berlin, Heidelberg, 1996, pp. 406–421.
[21] Cédric Fournet, Cosimo Laneve, Bisimulations in the join-calculus, Theor. Comput. Sci. 266 (1) (September 2001) 569–603.

[22] Simon J. Gay, Nils Gesbert, António Ravara, Session types as generic process types, in: Johannes Borgström, Silvia Crafa (Eds.), Proceedings Combined 21st International Workshop on Expressiveness in Concurrency, EXPRESS 2014, and 11th Workshop on Structural Operational Semantics, SOS 2014, Rome, Italy, 1st September 2014, in: EPTCS, vol. 160, 2014, pp. 94–110.
[23] Jean-Yves Girard, Linear logic, Theor. Comput. Sci. 50 (1) (January 1987) 1–101.
[24] Jean-Yves Girard, On the unity of logic, Ann. Pure Appl. Log. 59 (3) (February 1993) 201–217.
[25] Jean-Yves Girard, Yves Lafont, Linear logic and lazy computation, in: Hartmut Ehrig, Robert A. Kowalski, Giorgio Levi, Ugo Montanari (Eds.), TAPSOFT'87: Proceedings of the International Joint Conference on Theory and Practice of Software Development, Pisa, Italy, March 23-27, 1987, Volume 2: Advanced Seminar on Foundations of Innovative Software Development II and Colloquium on Functional and Logic Programming and Specifications (CFLP), in: Lecture Notes in Computer Science, vol. 250, Springer, 1987, pp. 52–66.
[26] Daniele Gorla, A taxonomy of process calculi for distribution and mobility, Distrib. Comput. 23 (4) (December 2010) 273–299.
[27] Kohei Honda, Olivier Laurent, An exact correspondence between a typed pi-calculus and polarised proof-nets, Theor. Comput. Sci. 411 (22) (May 2010) 2223–2238.
[28] Kohei Honda, Types for dyadic interaction, in: Eike Best (Ed.), CONCUR'93, in: Lecture Notes in Computer Science, Springer, Berlin, Heidelberg, 1993, pp. 509–523.
[29] Bas van den Heuvel, Jorge A. Pérez, Session type systems based on linear logic: classical versus intuitionistic, in: Stephanie Balzer, Luca Padovani (Eds.), Proceedings of the 12th International Workshop on Programming Language Approaches to Concurrency- and Communication-cEntric Software, Dublin, Ireland, 26th, April 2020, in: Electronic Proceedings in Theoretical Computer Science, vol. 314, Open Publishing Association, 2020, pp. 1–11.
[30] Bas van den Heuvel, Jorge A. Pérez, Deadlock freedom for asynchronous and cyclic process networks, in: Julien Lange, Anastasia Mavridou, Larisa Safina, Alceste Scalas (Eds.), Proceedings 14th Interaction and Concurrency Experience, Online, 18th, June 2021, in: Electronic Proceedings in Theoretical Computer Science, vol. 347, Open Publishing Association, 2021, pp. 38–56.
[31] Ross Horne, Luca Padovani, A logical account of subtyping for session types, Electronic Proceedings in Theoretical Computer Science 378 (April 2023) 26–37.
[32] Bas van den Heuvel, Joseph W.N. Paulus, Daniele Nantes-Sobrinho, Jorge A. Pérez, Typed non-determinism in functional and concurrent calculi, in: Chung-Kil Hur (Ed.), Programming Languages and Systems, in: Lecture Notes in Computer Science, Springer Nature, Singapore, 2023, pp. 112–132.
[33] Kohei Honda, Vasco T. Vasconcelos, Makoto Kubo, Language primitives and type discipline for structured communication-based programming, in: Chris Hankin (Ed.), Programming Languages and Systems, in: Lecture Notes in Computer Science, Springer, Berlin, Heidelberg, 1998, pp. 122–138.
[34] Wen Kokke, Fabrizio Montesi, Marco Peressotti, Better late than never: a fully-abstract semantics for classical processes, Proc. ACM Program. Lang. 24 (January 2019) 1.
[35] Naoki Kobayashi, Type systems for concurrent programs, in: Bernhard K. Aichernig, Tom Maibaum (Eds.), Formal Methods at the Crossroads. From Panacea to Foundational Support: 10th Anniversary Colloquium of UNU/IIST, the International Institute for Software Technology of the United Nations University, Lisbon, Portugal, March 18-20, 2002. Revised Papers, in: Lecture Notes in Computer Science, Springer, Berlin, Heidelberg, 2003, pp. 439–453.
[36] Naoki Kobayashi, Benjamin C. Pierce, David N. Turner, Linearity and the pi-calculus, ACM Trans. Program. Lang. Syst. 21 (5) (September 1999) 914–947.
[37] Dimitrios Kouzapas, Jorge A. Pérez, Nobuko Yoshida, On the relative expressiveness of higher-order session processes, Inf. Comput. 268 (October 2019) 104433.
[38] Olivier Laurent, Around classical and intuitionistic linear logics, in: Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science, LICS '18, ACM, New York, NY, USA, 2018, pp. 629–638.
[39] Sam Lindley, J. Garrett Morris, Talking bananas: structural recursion for session types, in: Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming, ICFP 2016, ACM, New York, NY, USA, 2016, pp. 434–447.
[40] Massimo Merro, Locality and polyadicity in asynchronous name-passing calculi, in: Jerzy Tiuryn (Ed.), Foundations of Software Science and Computation Structures, in: Lecture Notes in Computer Science, Springer, Berlin, Heidelberg, 2000, pp. 238–251.
[41] Robin Milner, Joachim Parrow, David Walker, A calculus of mobile processes, I, Inf. Comput. 100 (1) (September 1992) 1–40.
[42] Jorge A. Pérez, Luís Caires, Frank Pfenning, Bernardo Toninho, Linear logical relations for session-based concurrency, in: Helmut Seidl (Ed.), Programming Languages and Systems, in: Lecture Notes in Computer Science, Springer, Berlin, Heidelberg, 2012, pp. 539–558.
[43] Jorge A. Pérez, Higher-Order Concurrency: Expressiveness and Decidability Results, PhD thesis, University of Bologna, Italy, May 2010.
[44] Jorge A. Pérez, The challenge of typed expressiveness in concurrency, in: Albert Elvira, Ivan Lanese (Eds.), Formal Techniques for Distributed Objects, Components, and Systems, in: Lecture Notes in Computer Science, Springer International Publishing, Cham, 2016, pp. 239–247.
[45] Kirstin Peters, Comparing process calculi using encodings, Electronic Proceedings in Theoretical Computer Science 300 (August 2019) 19–38.
[46] Joseph W.N. Paulus, Jorge A. Pérez, Daniele Nantes-Sobrinho, Termination in concurrency, revisited, in: Santiago Escobar, Vasco T. Vasconcelos (Eds.), International Symposium on Principles and Practice of Declarative Programming, PPDP 2023, Lisboa, Portugal, October 22-23, 2023, ACM, 2023, pp. 3:1–3:14.
[47] Zesen Qian, G.A. Kavvos, Lars Birkedal, Client-server sessions in linear logic, Proc. ACM Program. Lang. 5 (ICFP) (August 2021) 62:1–62:31.
[48] Sangiorgi Davide, Locality and interleaving semantics in calculi for mobile processes, Theor. Comput. Sci. 155 (1) (February 1996) 39–83.
[49] Harold Schellinx, Some syntactical observations on linear logic, J. Log. Comput. 1 (4) (September 1991) 537–559.
[50] Davide Sangiorgi, David Walker, The Pi-Calculus: A Theory of Mobile Processes, Cambridge University Press, October 2003.
[51] Bernardo Toninho, Luis Caires, Frank Pfenning, Corecursion and non-divergence in session-typed processes, in: Matteo Maffei, Emilio Tuosto (Eds.), Trustworthy Global Computing, in: Lecture Notes in Computer Science, Springer, Berlin, Heidelberg, 2014, pp. 159–175.
[52] Kaku Takeuchi, Kohei Honda, Makoto Kubo, An interaction-based language and its typing system, in: Costas Halatsis, Dimitrios Maritsas, George Philokyprou, Sergios Theodoridis (Eds.), PARLE'94 Parallel Architectures and Languages Europe, in: Lecture Notes in Computer Science, Springer, Berlin, Heidelberg, 1994, pp. 398–413.
[53] Bernardo Toninho, Nobuko Yoshida, Interconnectability of session-based logical processes, ACM Trans. Program. Lang. Syst. 40 (4) (December 2018) 17.
[54] Vasco T. Vasconcelos, Fundamentals of session types, Inf. Comput. 217 (August 2012) 52–70.
[55] Bas van den Heuvel, Correctly Communicating Software: Distributed, Asynchronous, and Beyond, PhD thesis, University of Groningen, 2024.
[56] Philip Wadler, Propositions as sessions, in: Proceedings of the 17th ACM SIGPLAN International Conference on Functional Programming, ICFP '12, ACM, New York, NY, USA, 2012, pp. 273–286.